Rule Syntax
8 months ago

Rule SyntaxRule Syntax

This section describes the different rule syntax supported in the Reporting Engine.

NWDB Rule SyntaxNWDB Rule Syntax

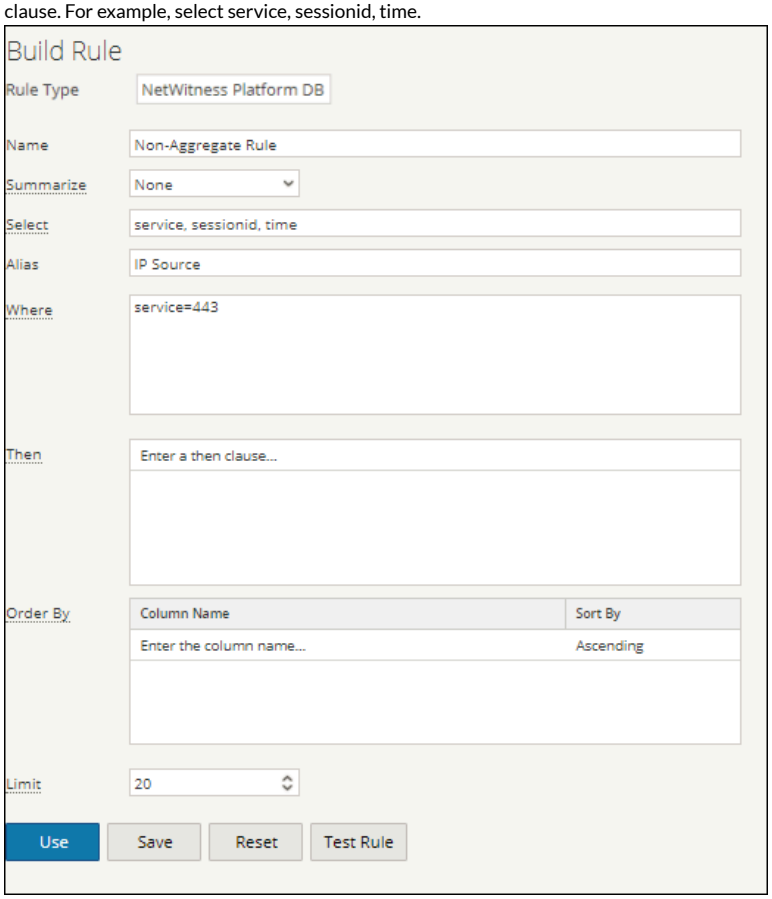
The NWDB rule is one of the rule syntax supported in the Reporting Engine. To enhance the execution time of your reporting entities, see "Reporting Guidelines" section in Reporting Overview (https://community.netwitness.com/s/article/ReportingOverview).
A Rule is a function that manipulates the result set of a rule in order to make the output in a report more meaningful or add additional functionality to a rule other than querying data and displaying it. Any combination of these rule actions can be used to create unique and interesting representations of the information collected by NetWitness.
The Reporting Engine supports the following categories of NWDB data source rule syntax:

- **select** clause
  - Non-Aggregate Rule
  - Aggregate Rule
- **alias**
- **where** clause
- **where** clause Operators
- **then** clause
- **Limit** field
- Rule Actions
- Rule Operators

Select Clause
The select clause is a comma separated list of values. For example: select sessionid,time,service.
There are two types of select clause for NWDB Rule:

- Non-aggregate rule
- Aggregate rule

Non-Aggregate Rule
When you want to define a rule without any grouping, choose 'None' in the Summarize field. In a non-aggregate rule, you can select any number of metas in the *Select* clause. For example, select service, sessionid, time.

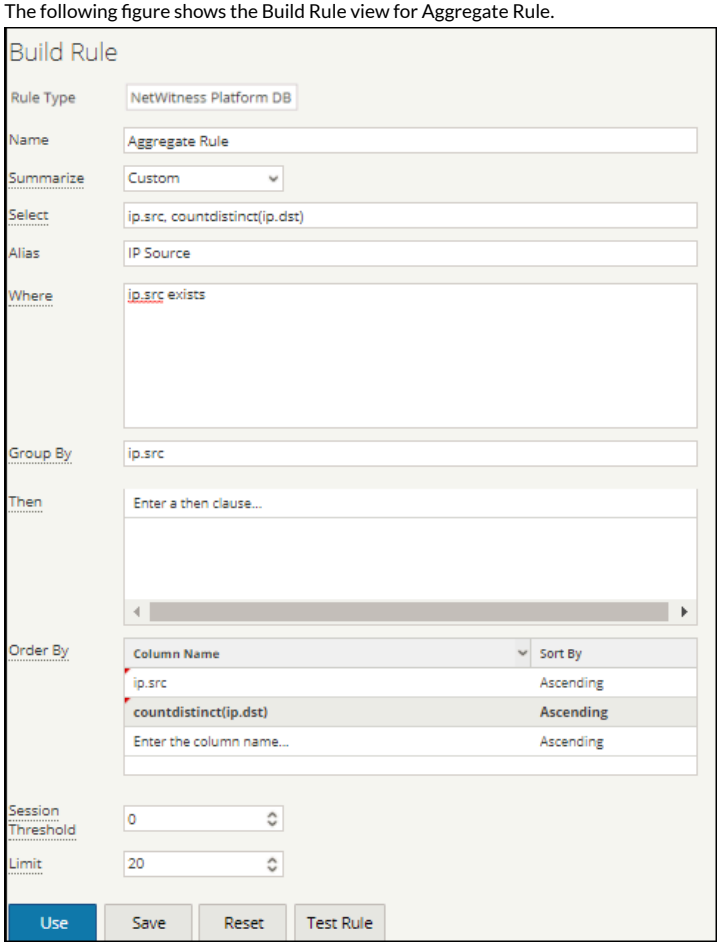


Aggregate Rule

When you want to query for a specific meta and its associated aggregate value then you must use the Aggregate rule. To get an aggregate, you must choose either of the three metas (Event Count, Packet Count, Session Size) or choose 'Custom' in the **Summarize** field to include an aggregate function in the *Select* clause. For example, select ip.src, sum (ip.dst). When Custom aggregate rule is enabled, the following fields are populated in the user interface:

- Group By
- Order By
- Session Threshold

The following figure shows the Build Rule view for Aggregate Rule.



There are two types of aggregate values that can be queried:

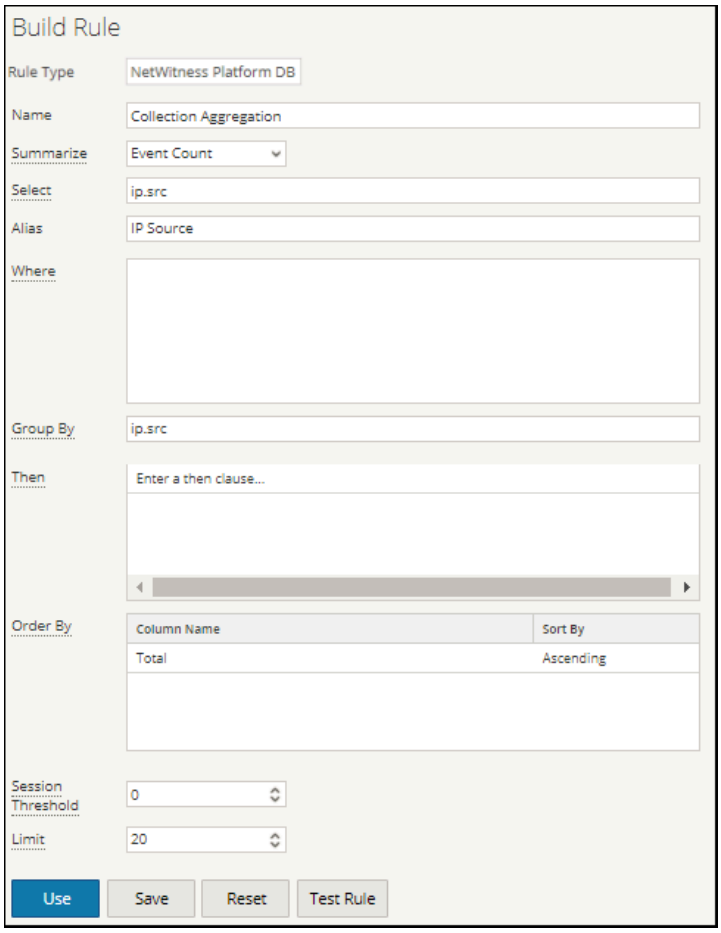
- Collection aggregation
- Meta aggregation

Collection Aggregation

With collection aggregation, you can get aggregates related to Event, Session or Packets. The following values can be queried in a collection aggregation:

- **Event Count**: The total count of events.
- **Packet Count**: The total count of packets.
- **Session Size**: The total session size.

These options are listed in 'Summarize' field and any one of them can be selected in a rule.
For example, choose any of the Collection aggregates (Event Count or Packet Count or Session Size) in the 'Summarize' field and select ip.src.



Meta aggregation

With meta aggregation, you can get aggregates of meta values. The following are the supported meta aggregate functions:

- sum(meta)
- count(meta)
- countdistinct(meta)
- min(meta)
- max(meta)
- avg(meta)
- first(meta)
- last(meta)
- len(meta)
- distinct(meta)

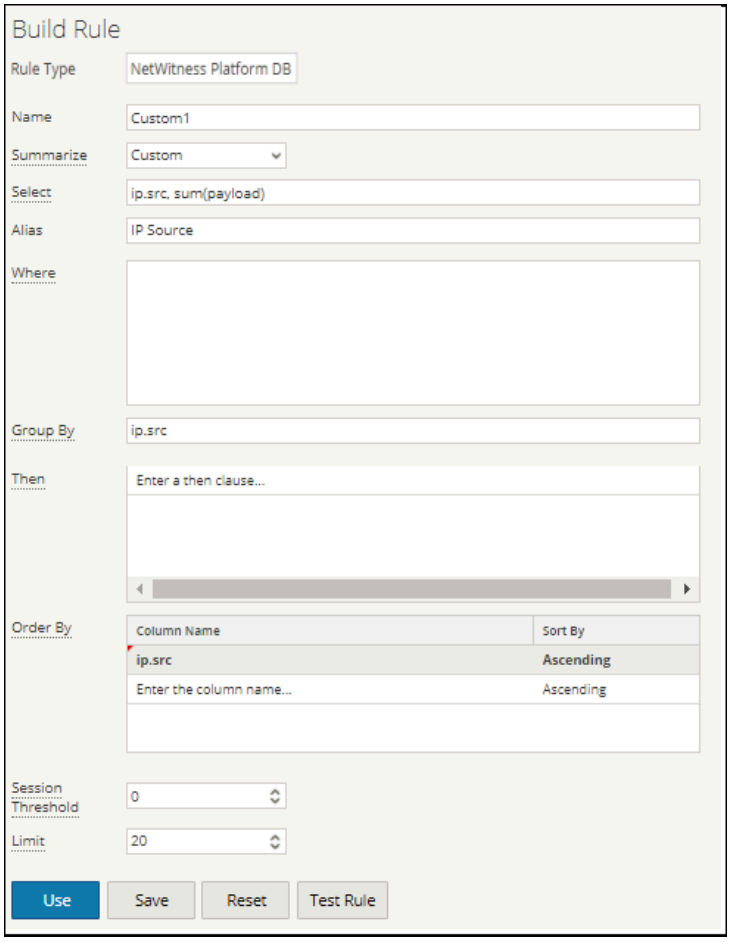
Supported Meta Aggregate Functions

The NWDB service supports the following meta aggregate functions and syntax in this release.

Syntax	Function
sum(<meta>)	The sum of all meta values. For example, if you provide the field sum(payload) in the select clause, the resultset is the sum of payload size. **Note:** The meta field chosen for the sum aggregate function must be of numeric data type.
count(<meta>)	The total number of meta fields that would be returned. For example, if you provide the field count(ip.dst) in the select clause, the resultset is the number of times an ip.dst value is returned.
countdistinct(<meta>)	The total number of distinct meta fields that would be returned. For example, if you provide the field countdistinct(ip.dst) in the select clause, the resultset is the number of times a distinct ip.dst value is returned.
min(<meta>)	The minimum of all meta values. For example, if you provide the field min(payload) in the select clause, the resultset is the min of payload size.
max(<meta>)	The maximum of all meta values. For example, if you provide the field max(payload) in the select clause, the resultset is the max of payload size.

Syntax	Function
avg(<meta>)	The average of all meta values. For example, if you provide the field avg(payload) in the select clause, the resultset is the avg of payload size. **Note:** The meta field chosen for the avg aggregate function must be of numeric data type.
first(<meta>)	The first occurrence of the meta value. For example, if you provide the field first(ip.src) in the select clause, the resultset is the first occurrence of ip.src for that group.
last(<meta>)	The last occurrence of the meta value. For example, if you provide the field last(ip.src) in the select clause, the resultset is the last occurrence of ip.src for that group.
len(<meta>)	Converts all field values to a UInt32 length instead of returning the actual value. This length is the number of bytes to store the actual value, not the length of the structure stored in the meta database. For instance, the meta value "NetWitness" returns a length of 10. All IPv4 fields, like ip.src, returns 4 bytes.
distinct(<meta>)	The distinct values of the meta. For example, if you provide the field distinct(ip.src) in the select clause, the resultset is all the distinct ip.src for that group.

You must select 'Custom' in 'Summarize' field and provide the meta and the meta aggregate functions in the select clause.



Note: Meta aggregate functions cannot be used in a WHERE clause and the rule actions like min_threshold/max_threshold can be used to filter aggregate functions. It is advised to use a more refined WHERE clause to get a better rule performance while using 'group by'.

Aggregate Query for Multiple Meta

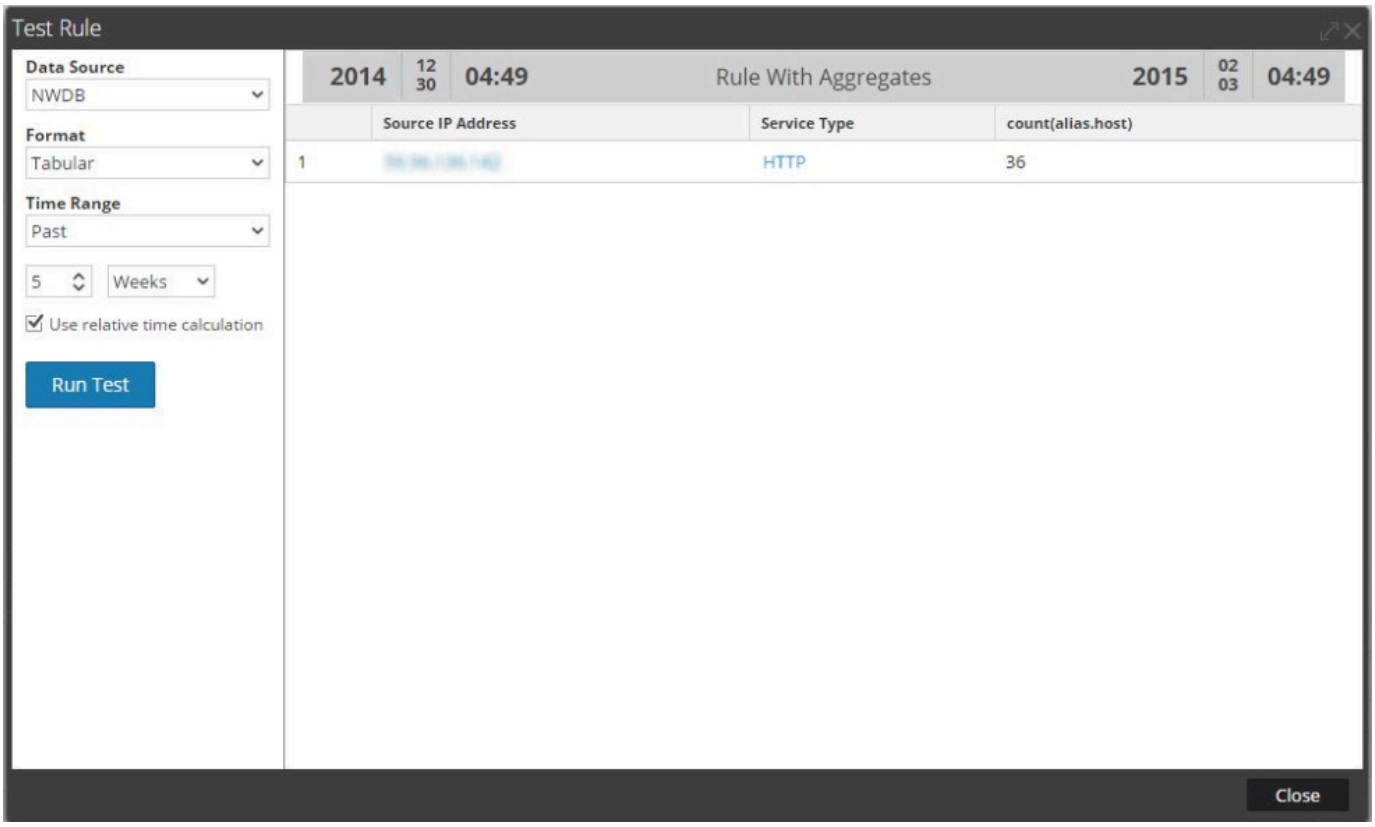
To execute aggregate query for multiple Meta, follow these steps:

1. Go to **Reports**.
   The Manage tab is highlighted and the **Rules** view is displayed.
2. In the **Rules** toolbar, click + > **NetWitness Platform DB**.

   For example, enter the following meta in the fields highlighted below:
   SELECT: ip.src, service, count(alias.host)
   ALIAS: Source IP Address, Service Type, count(alias.host)
   WHERE: ip.src = 59.96.136.142

Note: In the alias field you can enter a name for columns used in the select clause. If you do not specify the alias for one of the field in the select clause, then the default description will be used. For example, if the select clause has Field1, Field2, Field3, Field4, and alias has only Field1, Field3, Field4, then for Field2 a default description is used.

3. Click the **Test Rule** button at the bottom of the screen.
   The Test Rule page is displayed.



Summarize
Summarize determines the type of summarization or aggregation for the rule.

Name	Config Value
Summarize	To query metas without any custom grouping, select: • **None**: The data is grouped by session in this case. To get collection (sessions/events/packets) related aggregates, select either of the following: • **EventCount**: The total count of events. • **Packet Count**: The total count of packets. • **Session size**: The total session size. To get meta based aggregates, select: • **Custom**: This indicates that expected meta aggregate function is defined in rule select clause.

Order By
Order By determines how to sort the result set.

Name	Configuration Value
Column Name	The **Column Name** is the name of the columns by which you want to sort the results. By default, the value is empty. When you click on a column, the value gets populated based on the Summarize field. • For 'None' and 'Custom', the value gets populated based on the entries made in the Select field. You can select from this list or add custom name. • For Event Count, Packet Count and Session size, accepted values are Total and Value. • Total - sort by aggregate value • Value - sort by group by meta
Sort By	**Sort By** determines the order in which you want to sort the results. The following are the values: • Ascending Order • Descending Order

Session Threshold
The session threshold is the optimization setting to stop scanning the matching sessions for each possible unique value for the selected meta. The threshold is an integer between 0 (default) and 2147483647. The threshold 0 scans for all matching sessions.

Note: If you provide a non-zero value (a value higher than zero), the aggregate results are inaccurate. This can be used only when you are interested in unique values and not aggregate values.

Supported where Clause

Syntax	Description
where <field1> [<field-operator>] <value1>, <value2>,<value3-value4> <logic-operator> <field2>,and so on	The where clause is a comma separated list of language field values and ranges that is used by NwValues function. In the where clause, string values have to be enclosed within single quotes. For example, where username = 'admin' && service = 22.
where <field1> [<field-operator>] <List1>	You can use a list in the where clause if you have multiple values to report on. For example, where ip.src exists && alias.host exists && alias.host contains $[User Reports/List of Alias Host]. When you use the list you must specify in the format $[<path>/<List name>].

In the where clause, make sure the syntax is correct based on the meta type.

For example,

For all text meta type use quotes for example, username = 'user1'.

For all IP Addresses, Ethernet Addresses, and Numeric meta types do not use quotes for example, service = 80 && ip.src = 192.168.1.1.

For date and time meta types, if the date and time format is 'YYYY-MM-DD HH:MM:SS', use quotes.

If the date and time format is 1448034064 (number of seconds since EPOCH (Jan 1, 1970)), do no use quotes.

Note: If list is used in the rule, make sure that the list values are quoted or unquoted based on the type of the meta used. Checking the **Quotes will be inserted for all the values** checkbox in list definition page (for more information see, "Create Lists or List Groups" section in Configure a Rule (https://community.netwitness.com/s/article/ConfigureaRule)) would quote all the list values.

Supported where Clause Operators

Syntax	Description
=	Returns results where the field is equal to any provided value. For example, tcp.dstport = 21-25,110 returns session with TCP destination ports of 21, 22, 23, 24, 25, or 110.
!=	Returns results for fields that do not match the values specified. For example, eth.type !=0x0800 returns sessions outside of hex value (decimal value of 2048) that is all non-IP based protocols.
begins	Checks for a value at the beginning of a text or binary field.
contains	Searches a text or binary value for a partial match.
ends	Checks for a value at the end of a text or binary field.
exists	If the field value exists, regardless of value, the operation evaluates to true.
!exists	If the field value does not exist, the operation evaluates to true.
length	Evaluates the length of the field. For example, username length 20-u returns any username that is 20 or more characters long.
regex	Performs a regular expression search against text or binary values.
not	Not operator is used to negate a clause or condition. For example, (not(user.dst ends "$")) will not display values for user destination.

Supported then Clause

Syntax	Description
then <rule action>	The then clause contains a rule action that manipulates the original result set of a rule in order to make the output in a report more concrete or add additional functionality other than querying data and displaying it. For example, dedup (filename).

Limit field

This indicates the limit to be put on the query while fetching data from the database. If a result set is sorted by event count, packet count, or session size, the limit represents the top (or bottom) N values to be returned. If the result set is not sorted, the first N values are returned.

Rule Actions

The NWDB data source rule syntax supports the following rule actions:

- dedup
- filter_on
- filter_out
- lookup_and_add
- max_threshold
- min_threshold
- regex
- sum_count
- sum_values
- show_whats_new

dedup (string field)

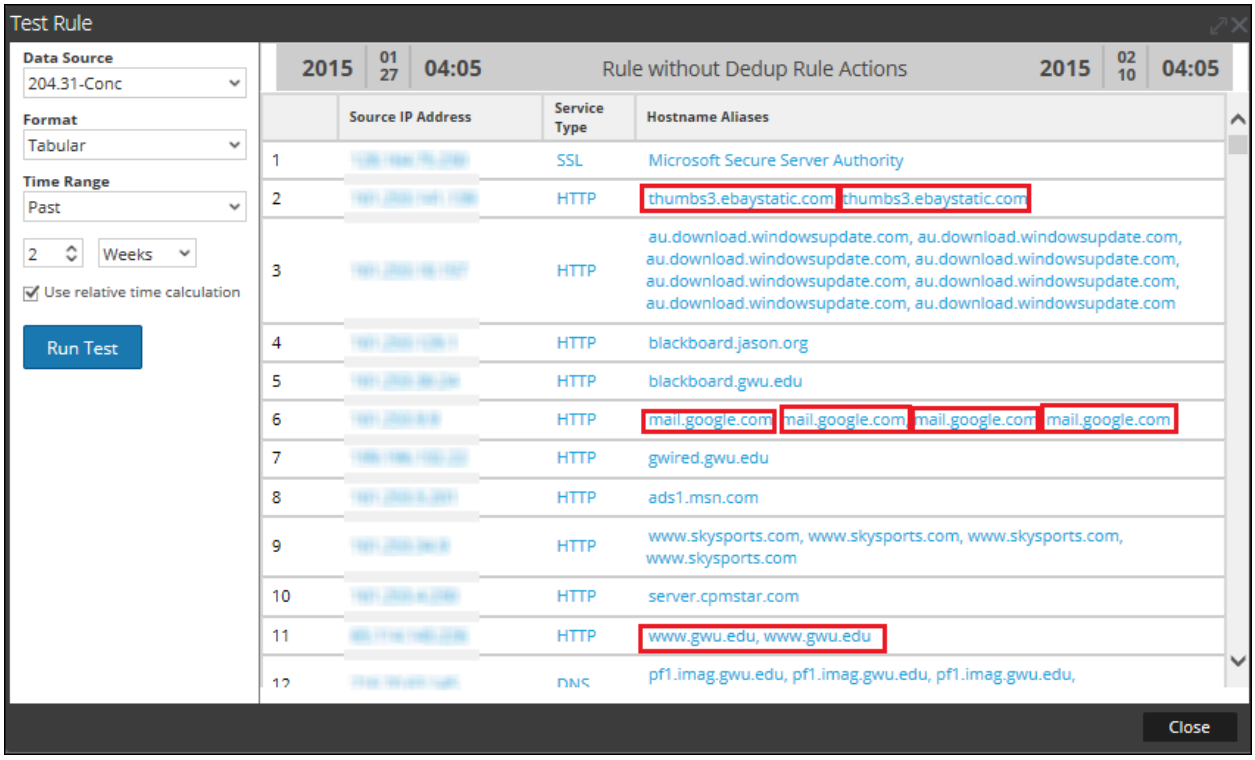
dedup removes the duplicate entries in an unsorted result set and displays only pertinent data. The dedup rule action removes duplicate entries of a specific field in the report, so that only the first occurrence of that value is listed in the report.

Note: The dedup rule action cannot be used with an aggregate rule.

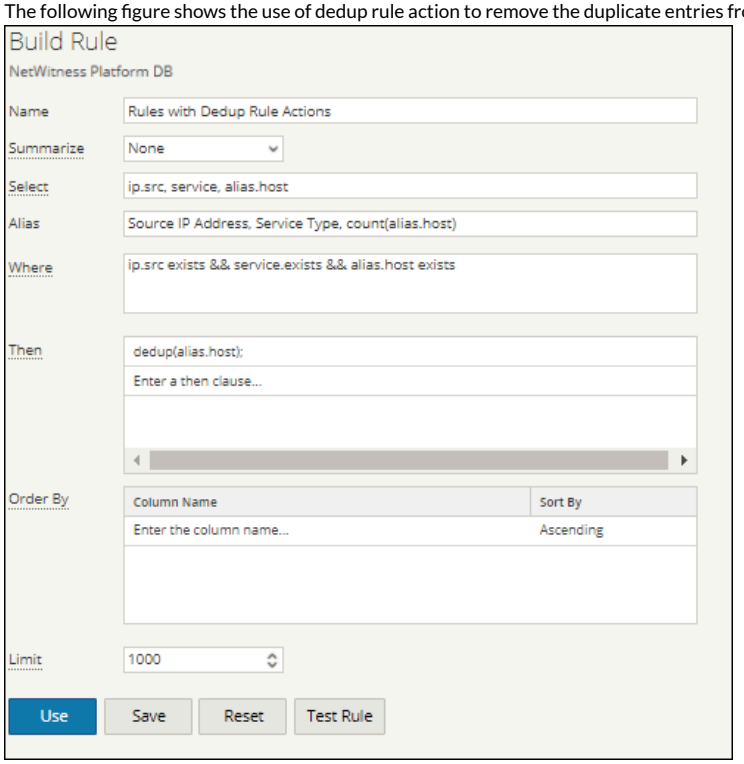
For example, the meta data generated by an individual session is often repetitive, especially when you have sessions with a lot of DNS lookups or web sessions that access the same host multiple times for various resources (such as, javascript, css). To remove the duplicate entries of the host, you can use the dedup rule action.

Example:

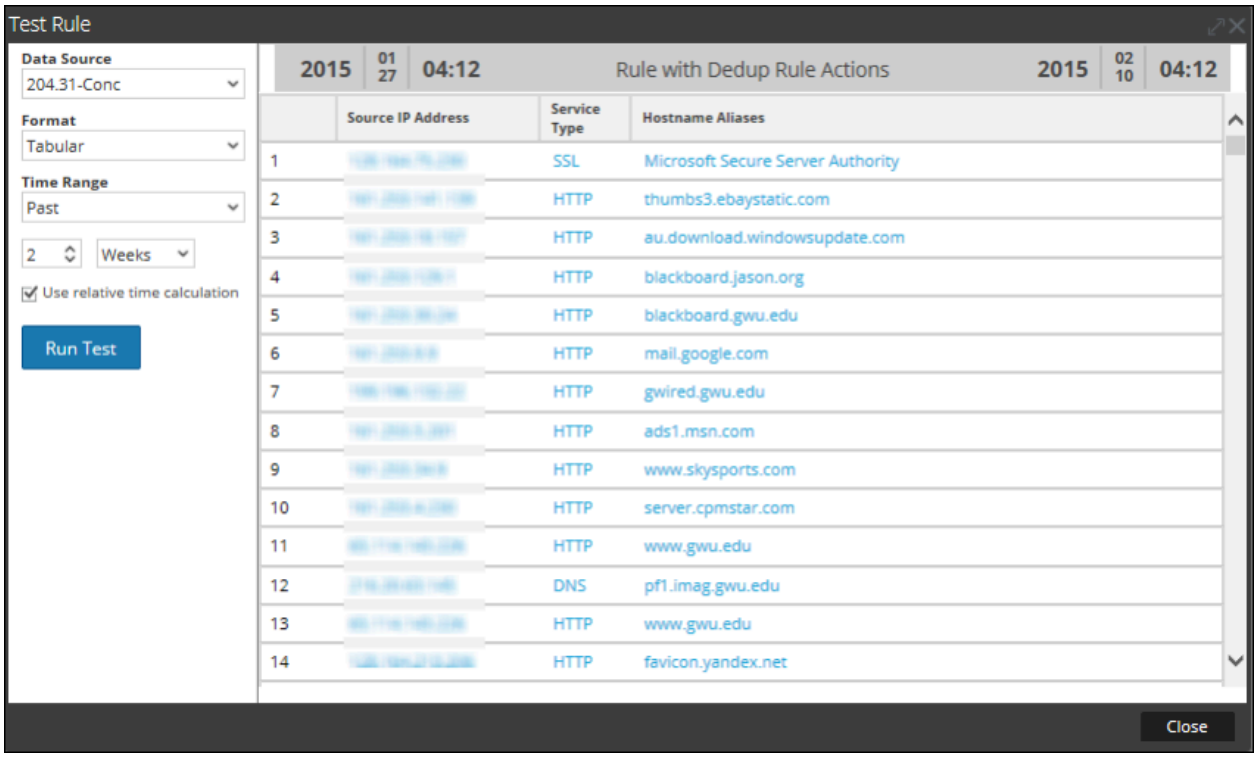
The following example is a lengthy result set that can be trimmed by removing the duplicate values in the same session.



The following figure shows the use of dedup rule action to remove the duplicate entries from the result set.



The duplicate value for each entry in the rule result set is reduced to one value.



filter_on (string filter, string field, bool matchExact)

filter_on removes values that do not contain the filter criteria from the result set. If the result set contains multiple fields, you must select a specific field to which the filter is applied. To add additional results to a single result set, include function such as lookup_and_add.

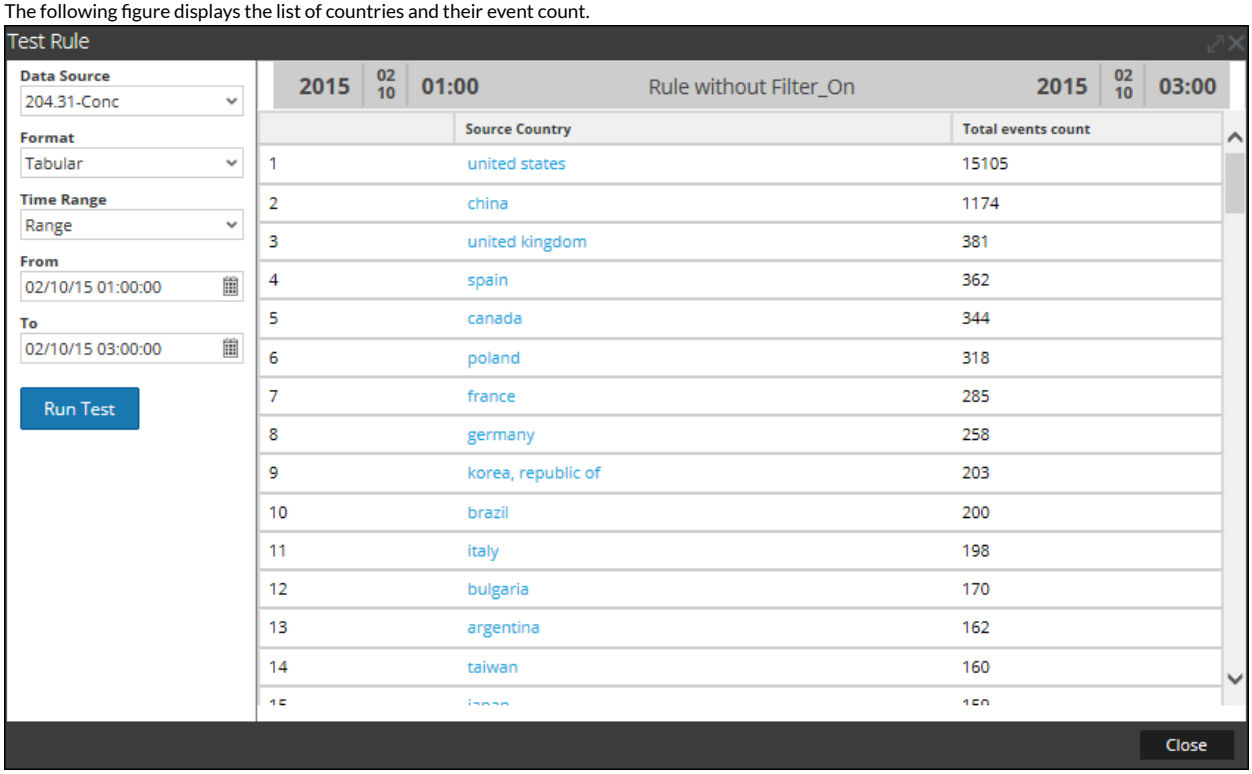
The matchExact parameter determines if the match is an exact match or contains a match.

- If matchExact is set to false, any value that contains the filter text is considered a match.
- If matchExact is set to true, only values that match the provided filter text is included in the result set.

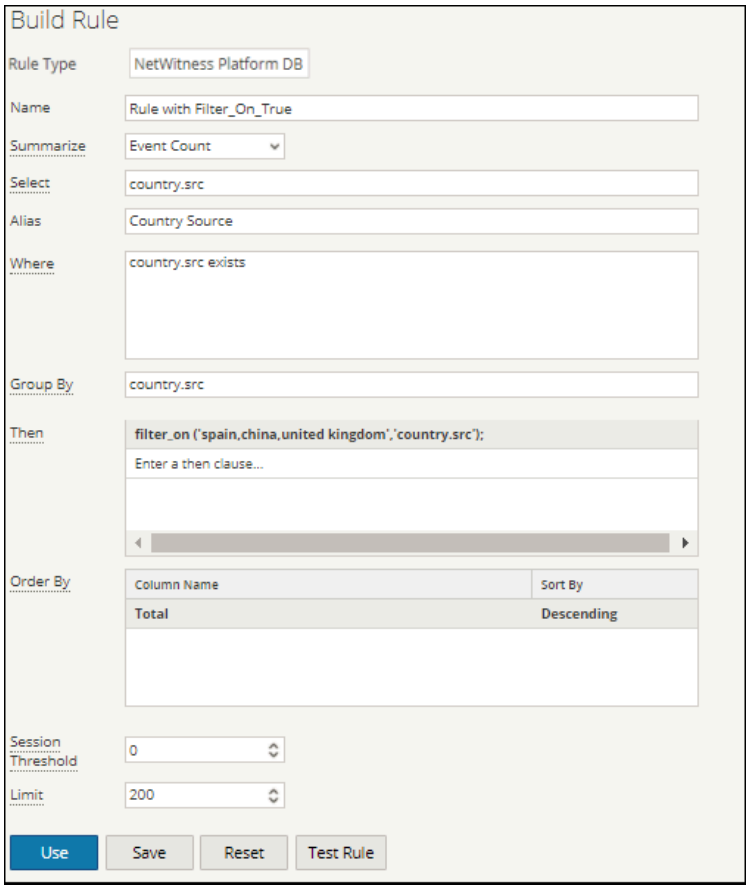
Note: Unless the matchExact parameter is specified, the default behavior of the rule action is to match exactly the text specified in the filter parameter. To specify that results containing the filter text must be kept in the result set, users must set the matchExact parameter to false.

Example:

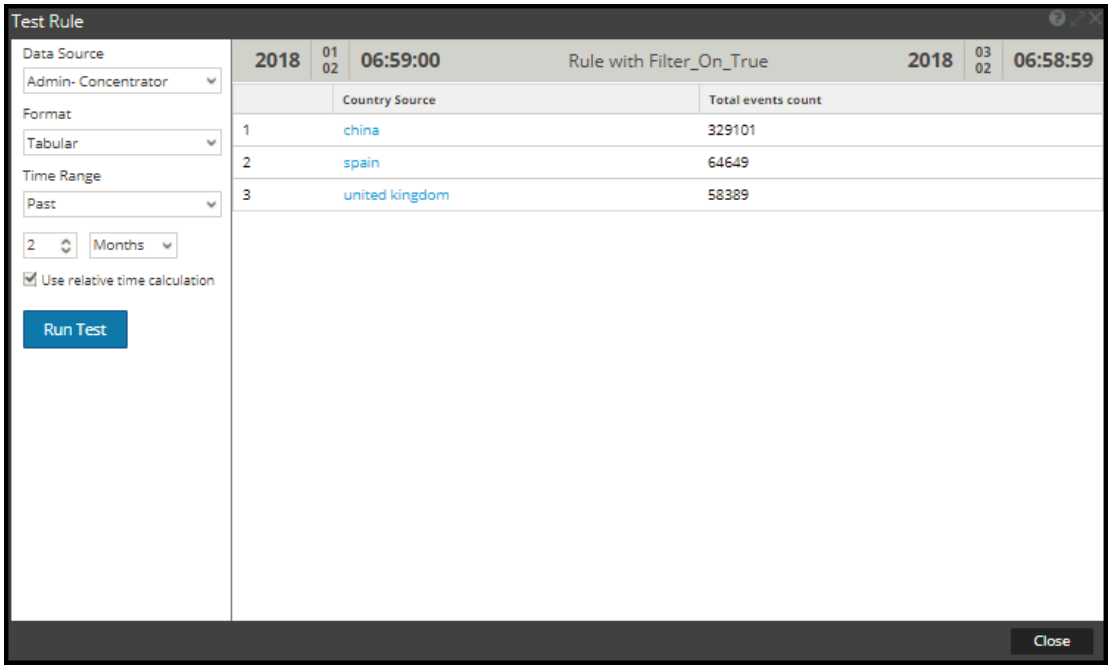
The following figure displays the list of countries and their event count.



The following figure shows a filter_on rule action to filter out countries except Spain, China, United States and United Kingdom from the result set.



The following figure shows the output with the filter_on rule action.

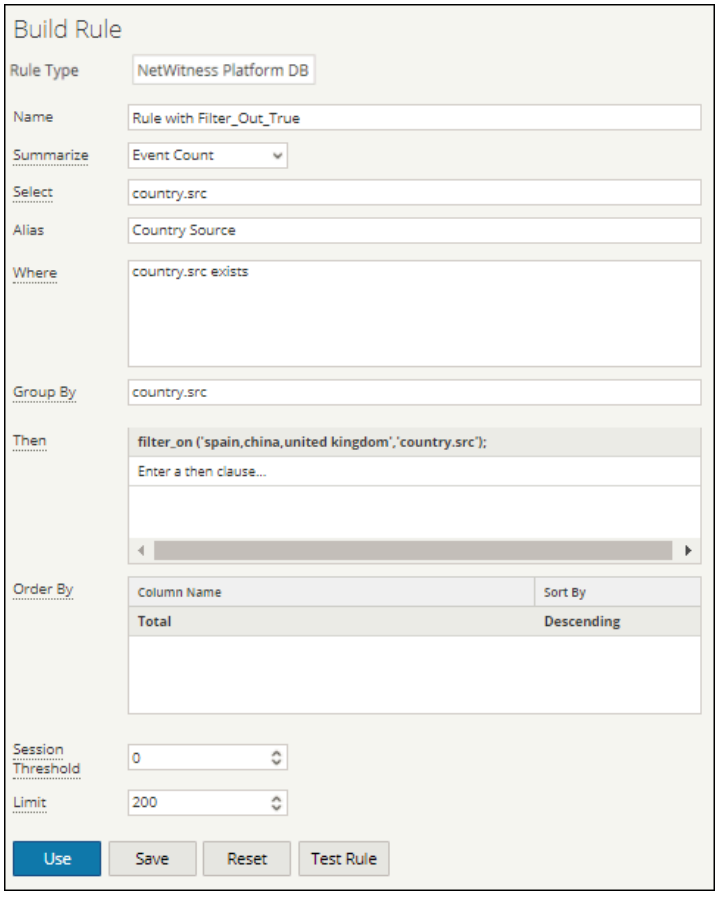


Another way of filtering out the entries from the result set is to create a list of variables which you want to filter out. For example, you can create a list with United Kingdom, France and Germany as values in the list. You can use this list in the rule action to get the same result set. For example, if you create a list called COUNTRY_LIST, you can use the list as follows:
filter_on ('$COUNTRY_LIST', 'country.src', 'false');
filter_out (string filter, string field)
filter_out (string filter, string field, bool matchExact)
filter_out removes the values that contain the *filter* criteria from the result set. If the result set contains multiple fields, you must select a specific field to which the filter is applied (for example, you can use a lookup_and_add to add results to a single result set).
The matchExact parameter determines if the match is an exact match or contains a match.

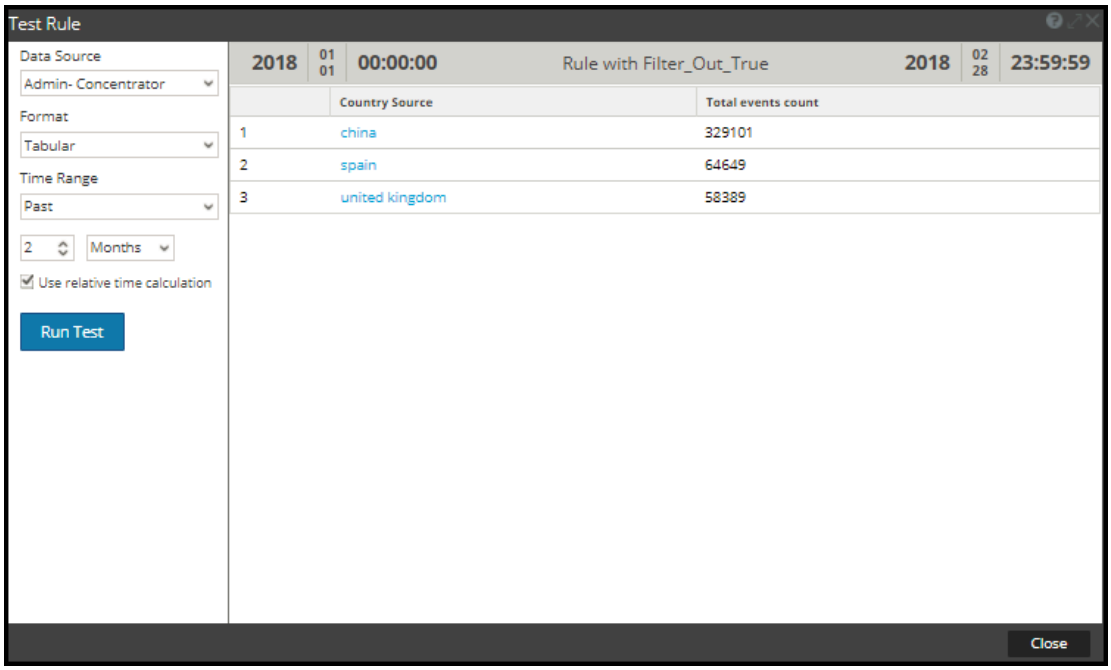
- If matchExact is set to false, any value that contains the filter text is considered a match.
- If matchExact is set to true, only values that match the provided filter text is excluded from the result set.

Note: Unless the matchExact parameter is specified, the default behavior of the rule action is to match exactly the text specified in the filter parameter. To specify that results containing the filter text must be removed from the result set, users must set the matchExact parameter to false.
Example:
The following figure displays the list of countries and their event count.

The following figure shows the filter_out rule action to remove the event count for Spain, China, United States and United Kingdom from the result set.



The following figure shows the output with the filter_out rule action.



lookup_and_add (string select, string field)
lookup_and_add (string select, string field, int limit)
lookup_and_add (string select, string field, int limit, boolean inherit)
lookup_and_add (string select, string field, int limit, boolean inherit, string extraWhere)
lookup_and_add(string select, string field, int limit, boolean inherit, string extraWhere, boolean aggregate)
This rule action iterates through a list of values in a result set and lookup additional meta data to further describe the relationships between various elements in a result set.
Note: The lookup_and_add rule action can be used only with an aggregate rule.
The first parameter, select, designates the type of meta data that must be added to elements of the result set. The second parameter, field, specifies where in the result set the append must apply to. Also, a limit can be applied to avoid crowding the result set with a large result set.
By default, subsequent queries to the SDK will inherit the where clause of the parent rule. To use a unique where clause, you can specify a boolean value in the fourth parameter as false and in the fifth parameter you can specify a different where clause.
Note: If you are using a unique where clause in your query, make sure that you use a single quote (') for enclosing arguments and double quotes (") for string values.
Now, with the addition of **Custom** summarization and **Group By** feature, the result can be achieved even without having lookup_and_add rule action. The new rule syntax with groupby displays the result in a flat structure which is better than the earlier rule syntax without groupby Hence it is recommended to manually edit/update rules with lookup_and_add rule action and use groupby clause wherever it is applicable.
Note: Lookup_And_Add rule action is supported only if the SELECT clause has one meta and aggregate function.
For example, see below scenarios: In Example **2a**, lookup_and_add rule action is used. Instead of using lookup_and_add rule action, the same result can be achieved by using **Custom** summarization and **Group By** feature. See Example **2b** below.
But, lookup_and_add rule action is still supported for NWDB rules on the following conditions:

- All versions of NWDB rules with Summarization as Event Count, Packet Count, or Session Size.
- For Custom summarization, the lookup_and_add rule must have only one group by meta with only one aggregate function where the aggregate function must be either sum() or count().

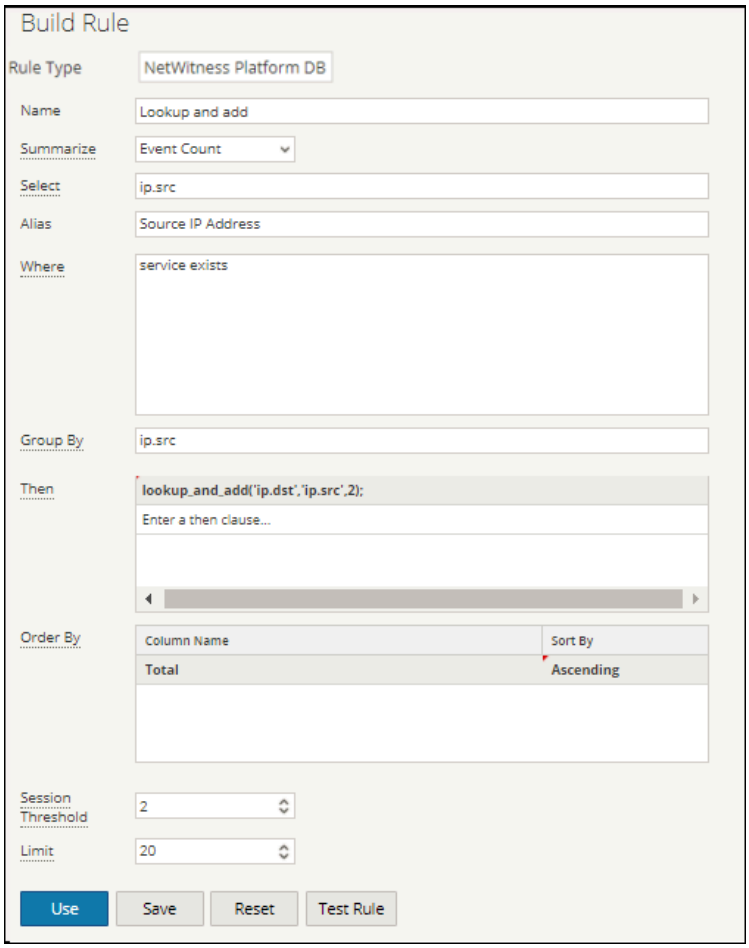
Note: It is not supported for “Summarize-None”.
For example, lookup_and_add rule action can be used for the following rules:

- select ip.src, sum(size) group by ip.src
- select ip.src, count(filename) group by ip.src

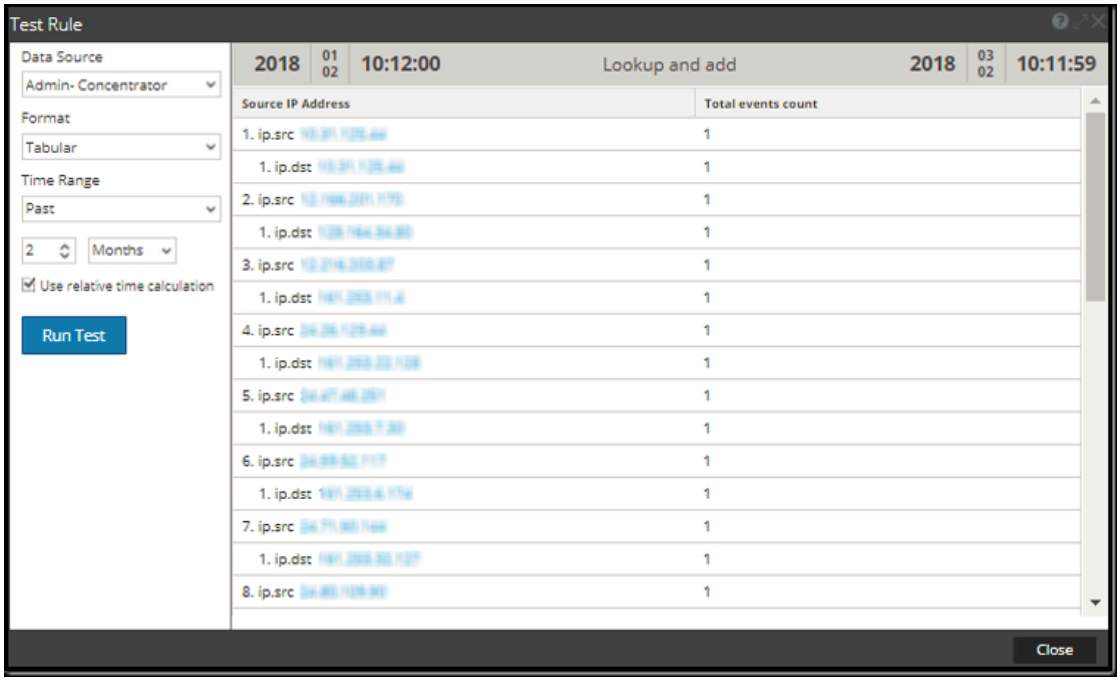
It cannot be used for the following rules:

- select ip.src, sum(size),count(filename) group by ip.src
- select ip.src, sum(size),avg(size) group by ip.src
- select ip.src,ip.dst count(filename) group by ip.src,ip.dst

Examples:
1. lookup_and_add('ip.dst','ip.src', 2);
This rule action would iterate through each ip.src in the initial result set and lookup the top two destination IP addresses with each ip.src.
The following figure shows the rule definition.



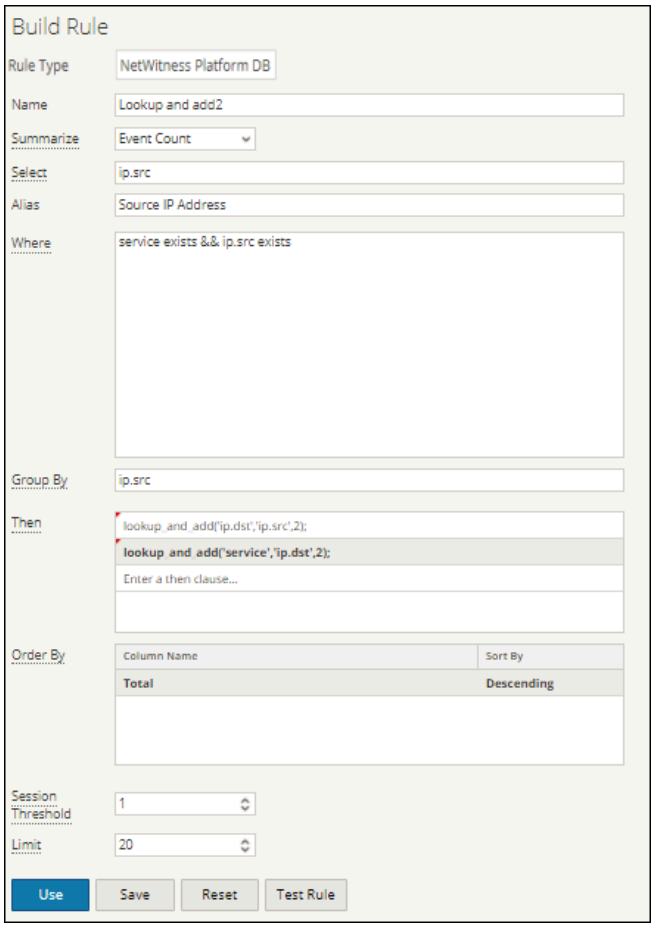
The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src.



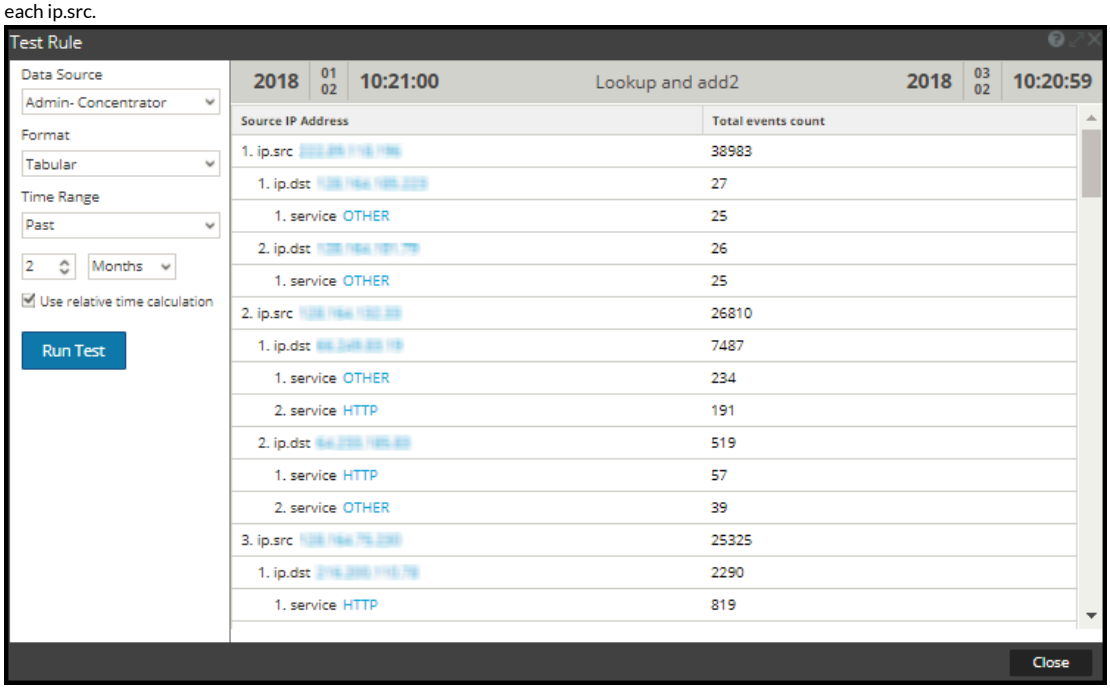
2a. lookup_and_add('ip.dst','ip.src', 2); lookup_and_add('service','ip.src', 3);

This rule action would iterate through each ip.src in the initial result set and lookup the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.

The following figure shows the rule definition.



The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.

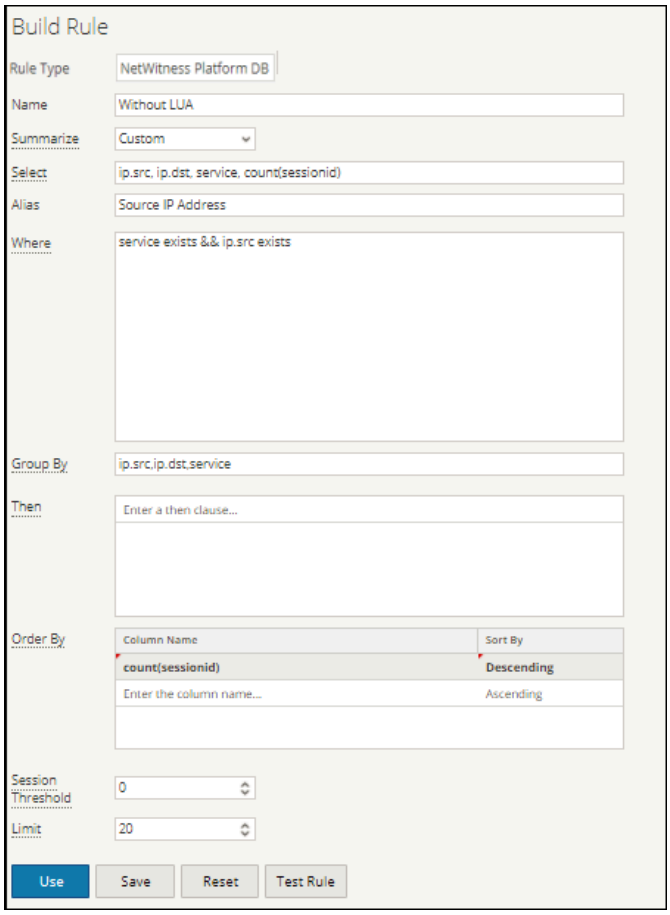


You can make the query as complex as you want by selecting different fields in the result set and by appending to different parts. For example, you may want to know what files each source IP had touched. However, because the parent rule has a WHERE clause of service = 6667 and the default behavior of this rule action is to append to the original WHERE clause, it becomes necessary to override the parent WHERE clause. The easiest way to understand this concept is to look at the previous lookup_and_add call lookup_and_add('ip.dst','ip.src',2). The actual query that is sent to the server is SELECT ip.dst WHERE service = 6667 &&ip.src = 206.42.199.194. In order to force the WHERE clause to override the service = 6667 portion of the WHERE clause (inherited from the parent rule), the user can specify a 4th parameter of false as shown in example 3.

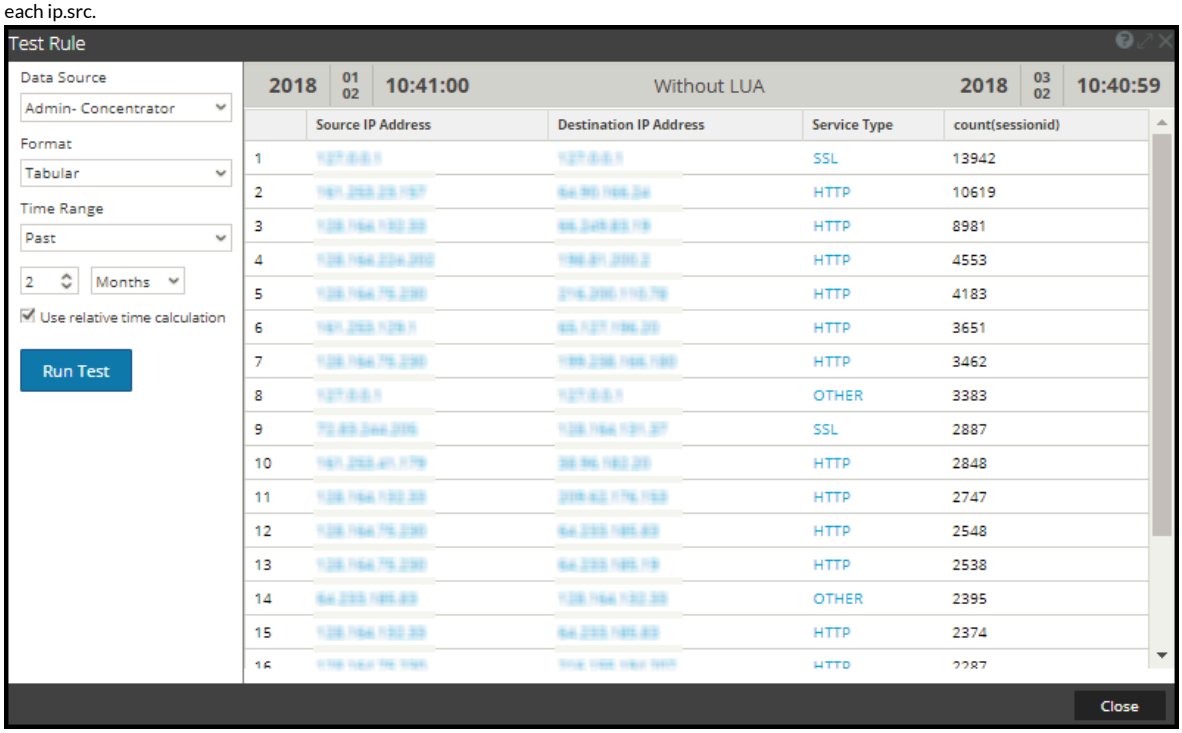
2b. Without Lookup_and_add Rule

This rule uses the Custom summarization and Group By feature to sort the results.

The following figure shows the rule definition.



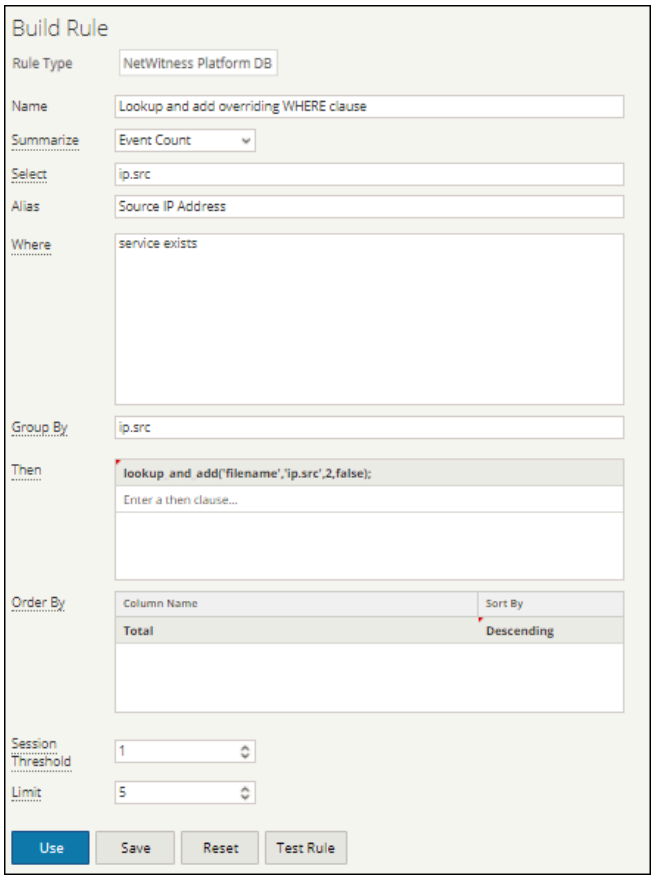
The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.



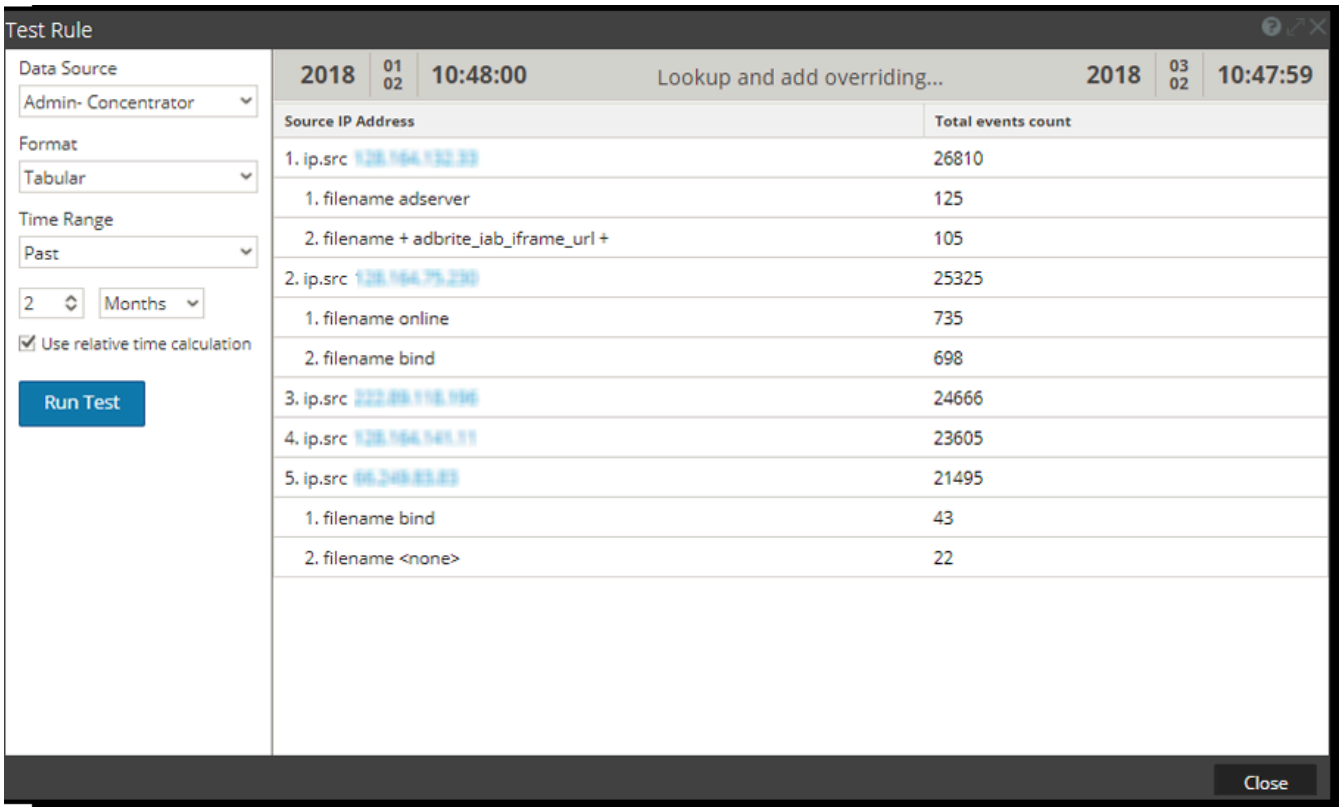
3. lookup_and_add('filename', 'ip.src', 2, false);

This call would issue a query to the server, like SELECT filename WHERE ip.src = 90.0.0.142 rather than SELECT filename WHERE service = 6667' && ip.src = 90.0.0.142 because you have specified the rule action to ignore the initial WHERE clause of the parent rule.

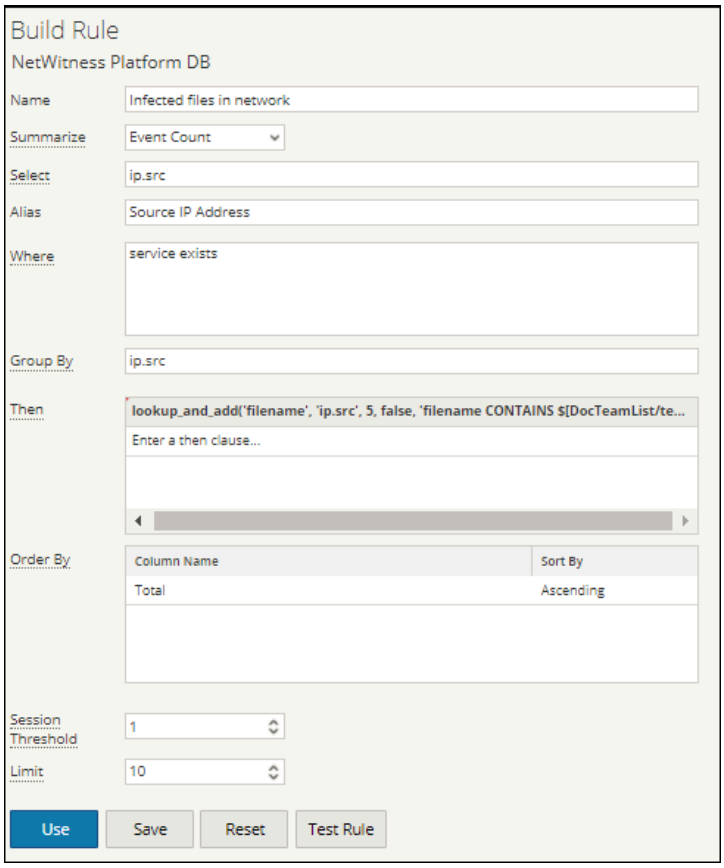
The following figure shows the rule definition.



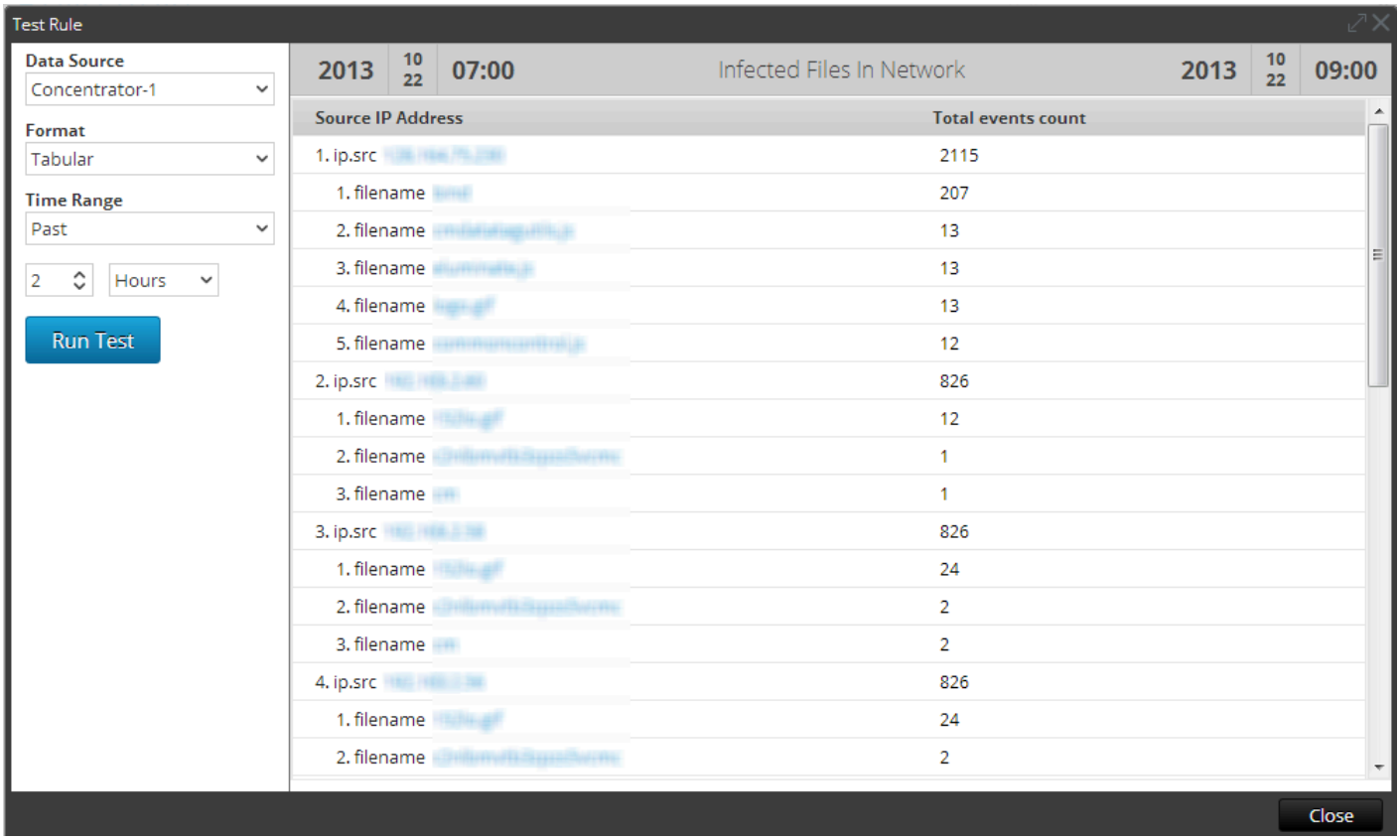
The following figure shows the result set.



The test list is in a group name netwitness, you can access that list with the following syntax.
You can even narrow down these appended results even further to only include filenames that have .gif as filename extension by using the fifth parameter in the rule action. The fifth parameter allows you to specify additional WHERE clause criteria. The files with .gif filename extension would be stored in the **test** list within a group named **DocTeamList**. You can access this list with the following syntax: threat.source = $[DocTeamList/test]
This can be referenced in the extra where clause parameter in the following manner:
4. lookup_and_add('filename', 'ip.src', 5, false, 'filename CONTAINS $[DocTeamList/test]');
The following figure shows the rule definition.



The following figure shows the result set.



5. lookup_and_add('ip.dst','ip.src', 2,true,,false);

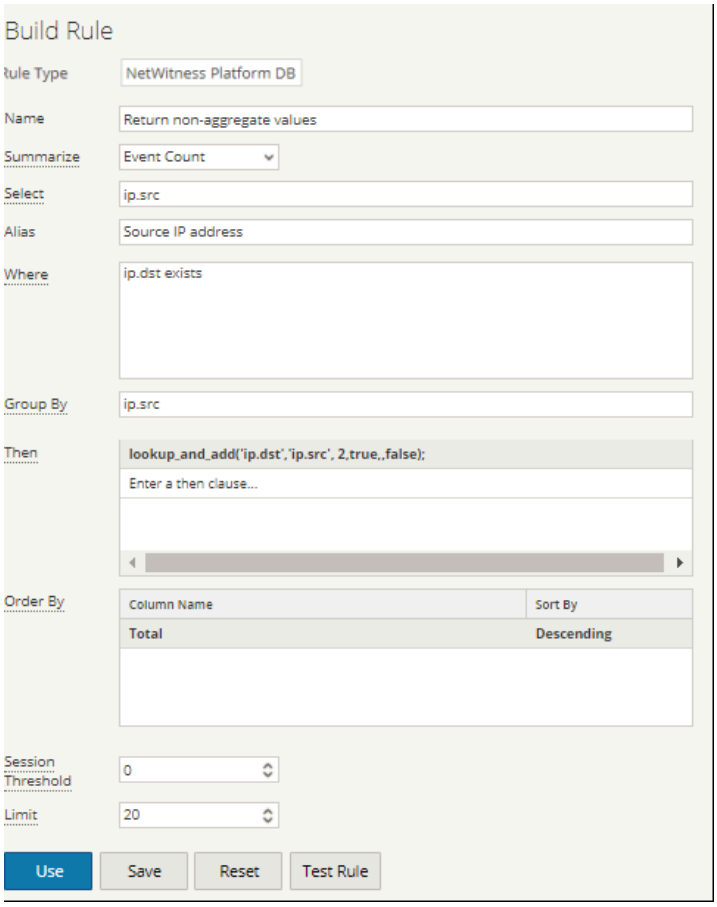
This rule action would iterate through each ip.src in the initial result set and lookup the top two destination IP addresses with each ip.src. The 'aggregate' parameter is set to 'false', this implies that aggregates would be skipped for lookup values and hence the lookup query executions will complete faster.

Note:

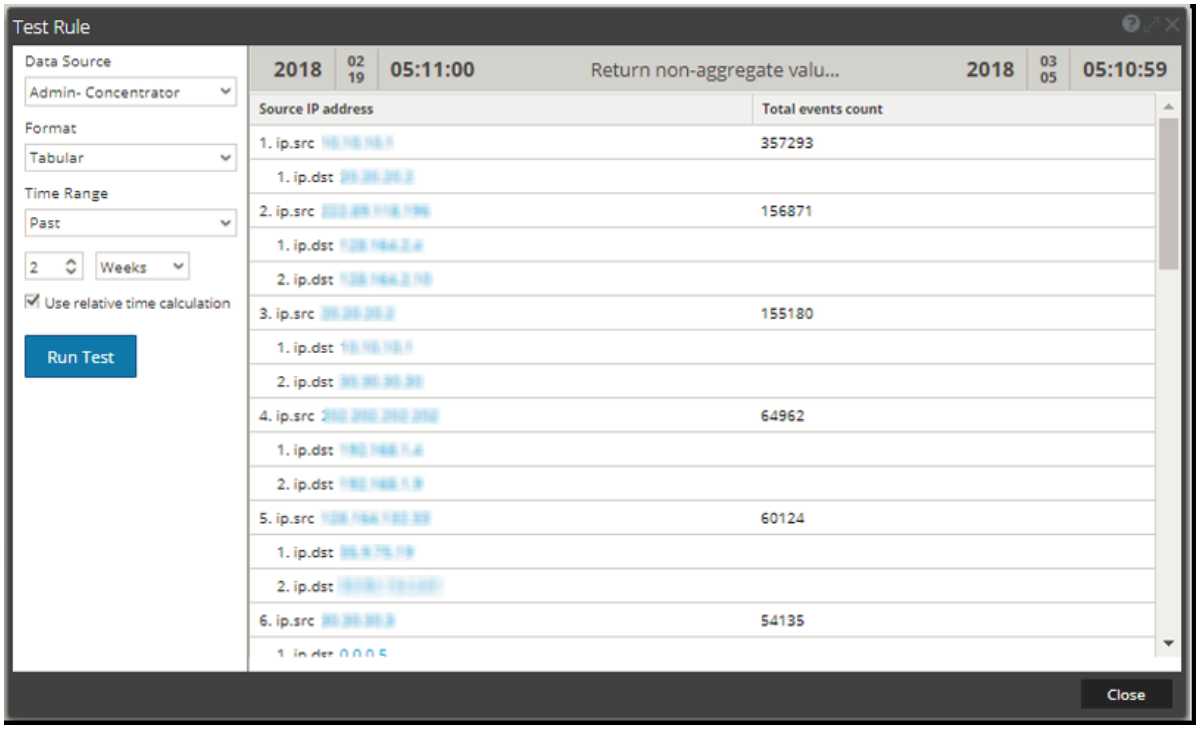
The default value for 'aggregate' is 'true'. When 'aggregate' is set to 'false', Reporting Engine passes threshold=1, Sort by='value' and Order=Ascending to NWDB to make lookup queries run faster.

. You must set the 'aggregate' to false, when rule contains aggregate functions or when the rule is run against a wide time range. This helps the rule to complete the execution faster.

The following figure shows the rule definition.



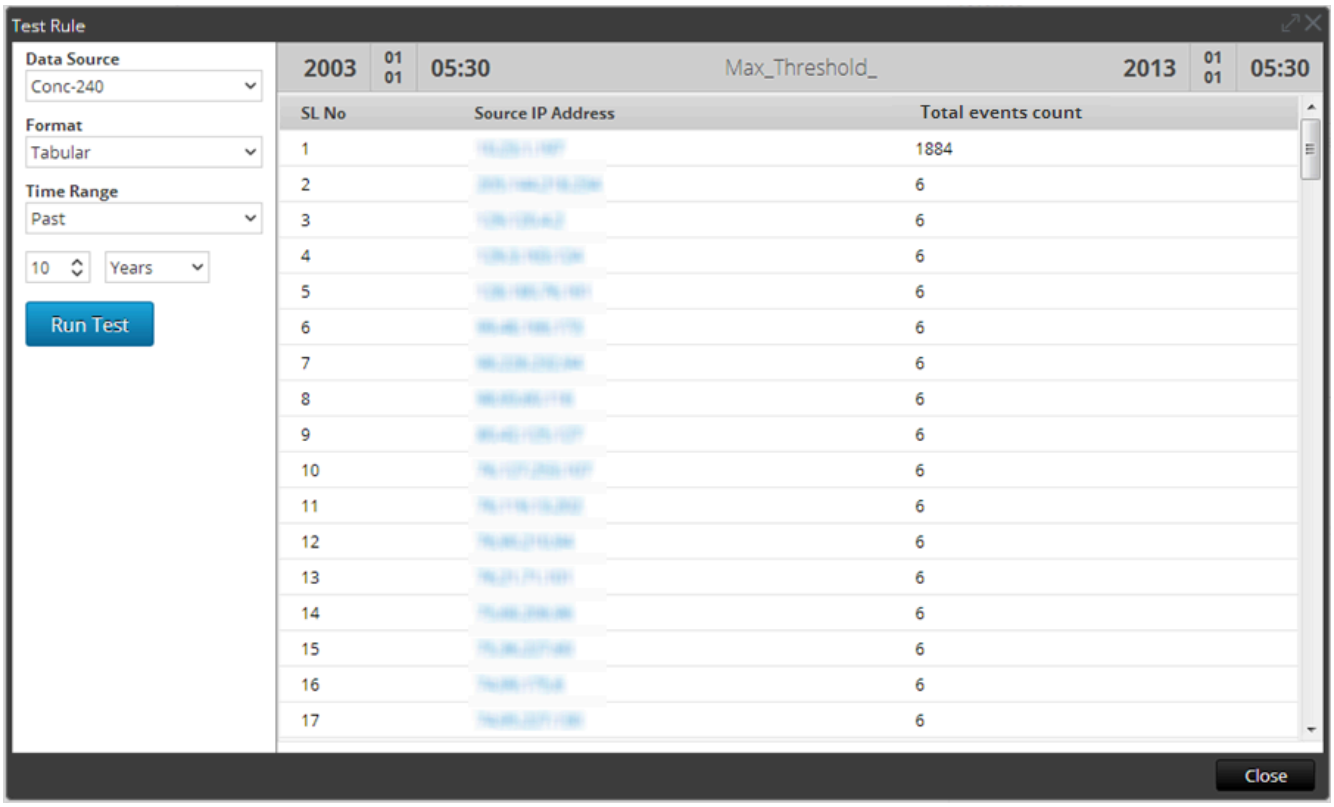
The following figure shows the result set.



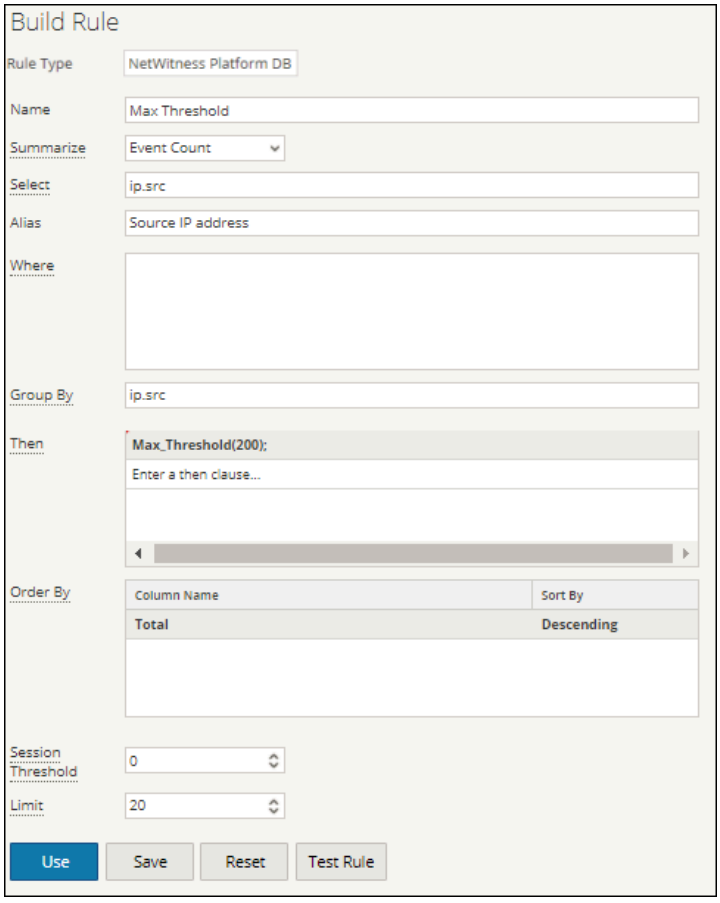
max_threshold (string quantity)
max_threshold (string quantity, string field)
max_threshold removes any results with a quantity that is larger than the maximum threshold quantity from a result set. The quantity can either be in terms of count or size and it is relative to the sorting options of the parent rule. This means that if you sort a rule by size, the rule action expects you to specify the parameter in bytes (you can append KB, MB, GB, TB to the parameter to make size conversion easier).
max_threshold rule can also be used to filter values based on the aggregate function values. Use the syntax based on the type of summarization used in the rule as below:

- max_threshold(String quantity): Can be used to filter Event Count, Packet Count, and Session Size.
- max_threshold(String quantity, String field): Can be used to filter values of Custom aggregates or any metas.

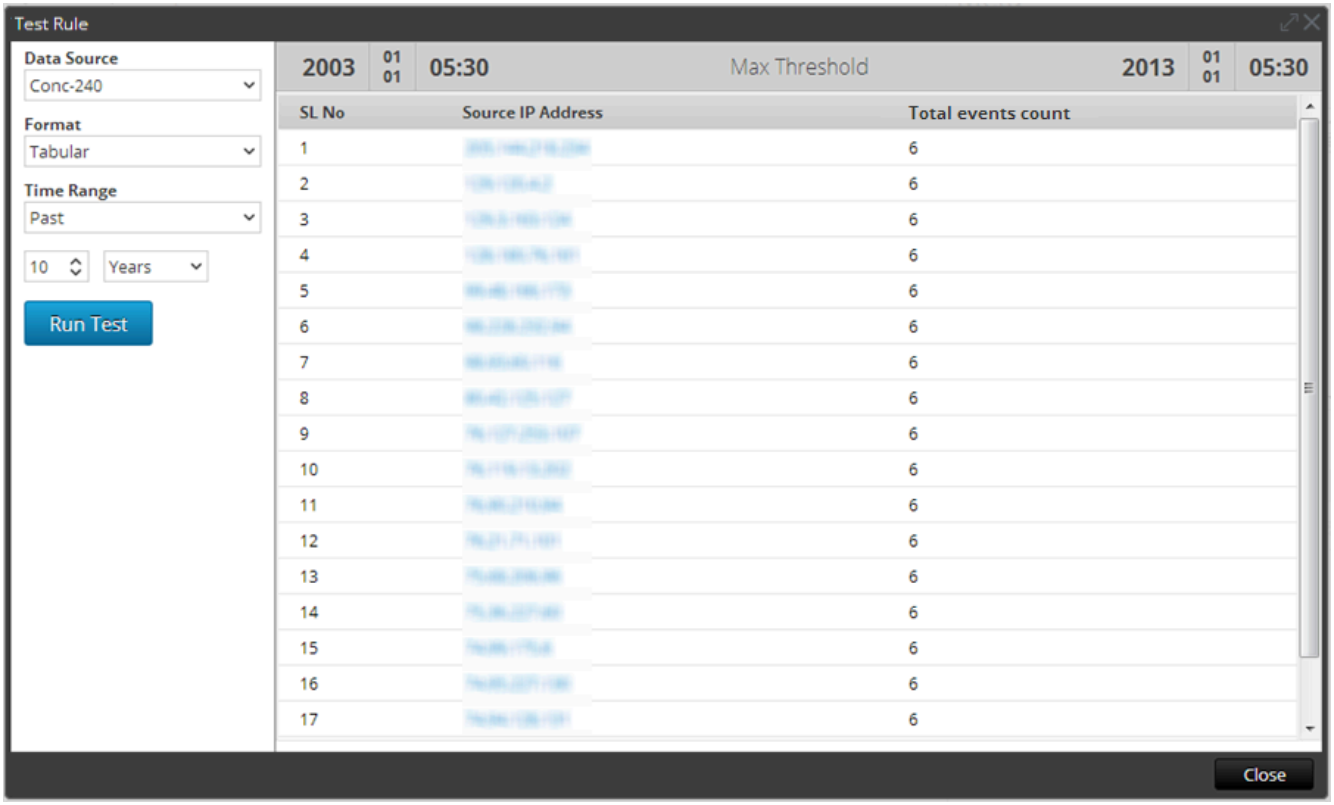
Examples:
1. max_threshold(200);
The following figure shows the result without the max_threshold argument. The output results have event counts exceeding 200.



The following figure shows a the max_threshold rule action that puts a limit of 200 bytes on the output. Any output having more than 200 bytes of data are not listed.



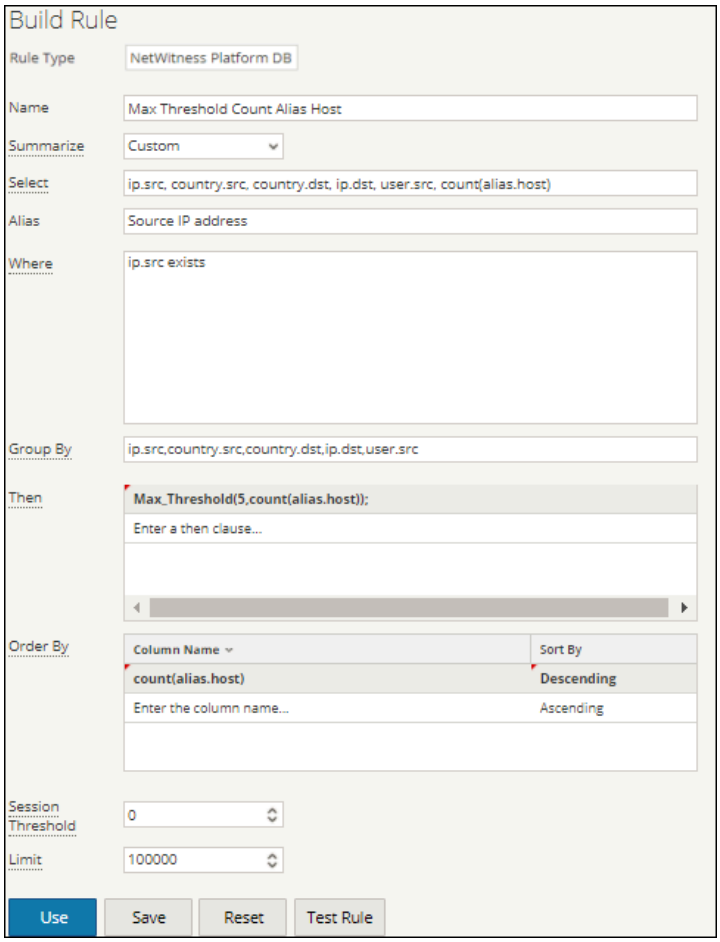
The following figure shows the result when the max_threshold rule action is applied. The result numbered 1 in the above screen capture is removed from the result.



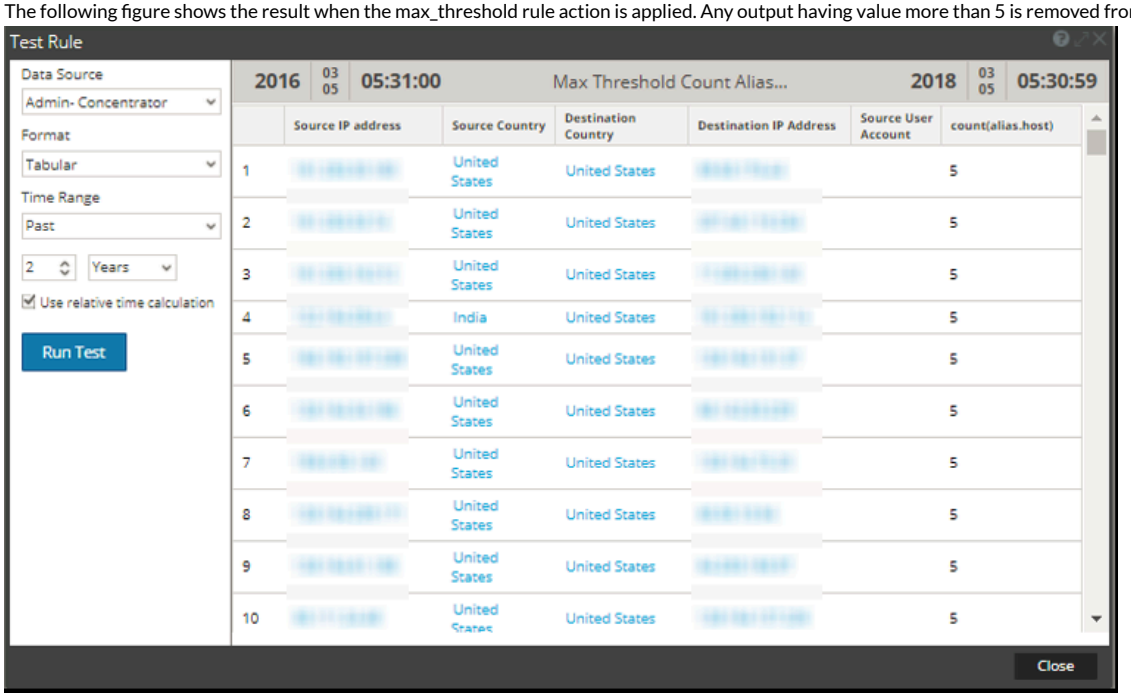
2. max_threshold(5,count(alias.host));

The following figure shows the result without the max_threshold argument. The output results have count of alias.host exceeding 5.

The following figure shows a the max_threshold rule action that puts a limit of 5 on the output. Any output having value more than 5 is not listed.



The following figure shows the result when the max_threshold rule action is applied. Any output having value more than 5 is removed from the result.



min_threshold (string quantity)

min_threshold removes results with a quantity that is smaller than the minimum threshold quantity from a result set. The quantity can either be in terms of count or size and it is relative to the sorting options of the parent rule. This means that if you sort a rule by size, the rule action expects you to specify the parameter in bytes (you can append KB, MB, GB, TB to the parameter to make size conversion easier).

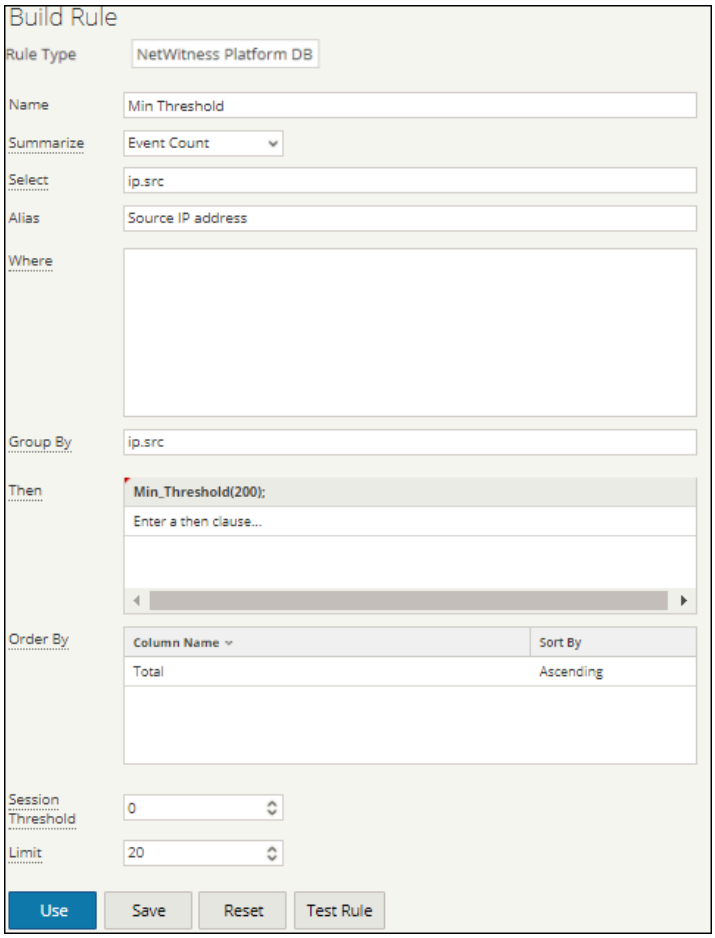
min_threshold rule can also be used to filter values based on the aggregate function values. Use the syntax based on the type of summarization used in the rule as below:

- min_threshold(String quantity): Can be used to filter Event Count, Packet Count, and Session Size.
- min_threshold(String quantity, String field): Can be used to filter values of Custom aggregates or any metas.

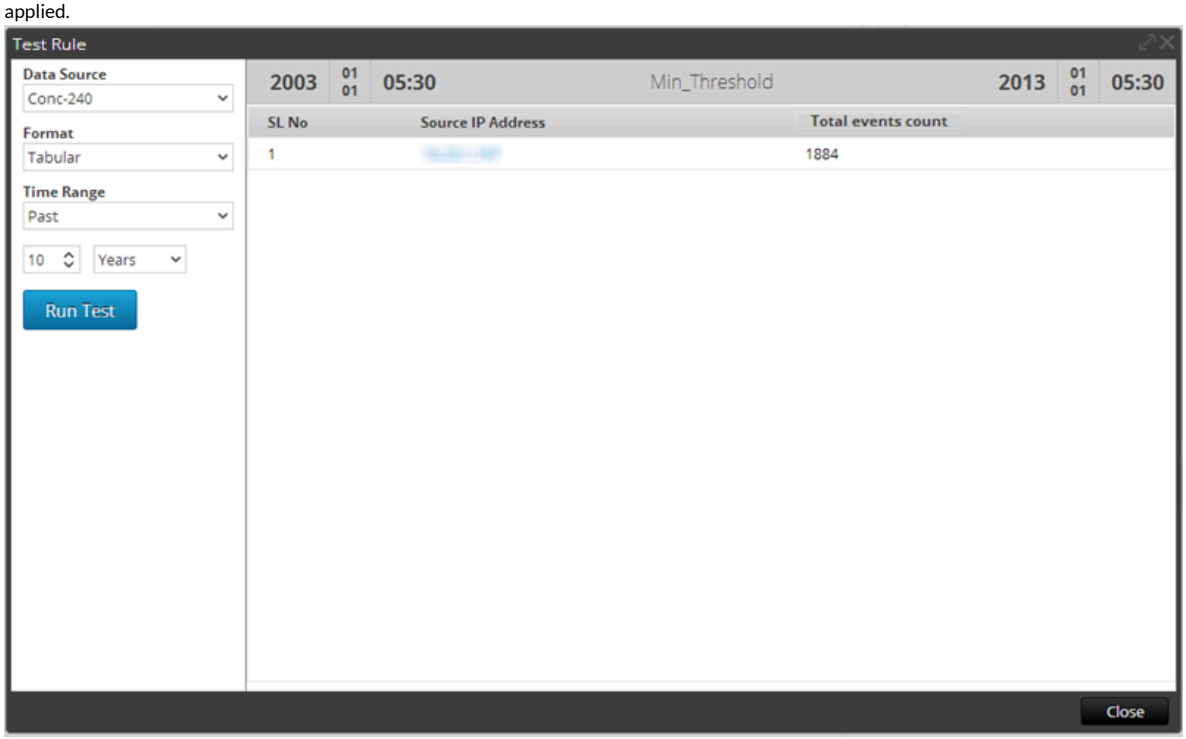
Examples:

1. min_threshold(200);

The following figure shows a sample of the min_threshold query.



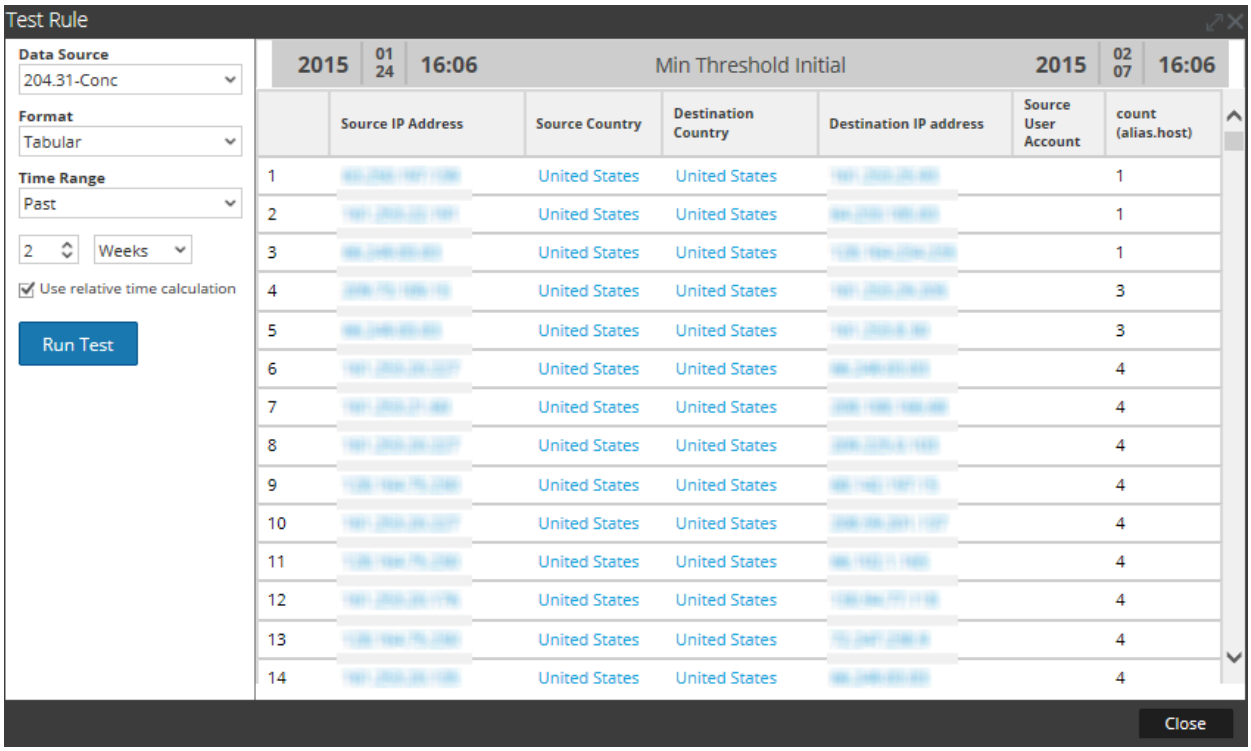
The above figure puts a limit of 200 bytes on the output. Any output having less than 200 bytes of data is not listed. The output with the min_threshold rule action is applied.



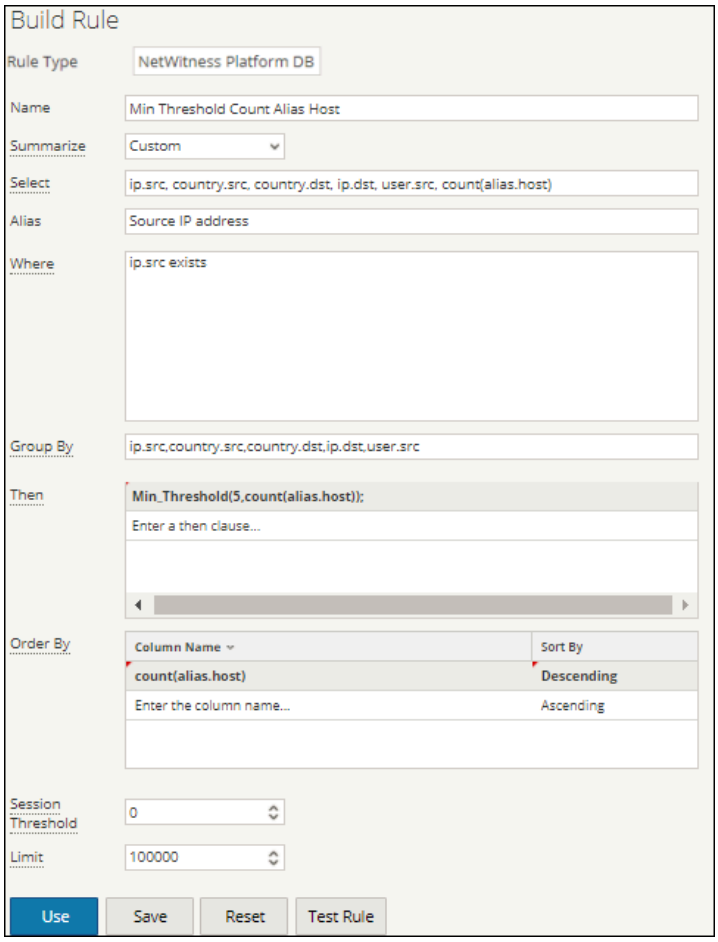
As shown, all the values are greater than 200 bytes.

2. min_threshold(100,count(alias.host));

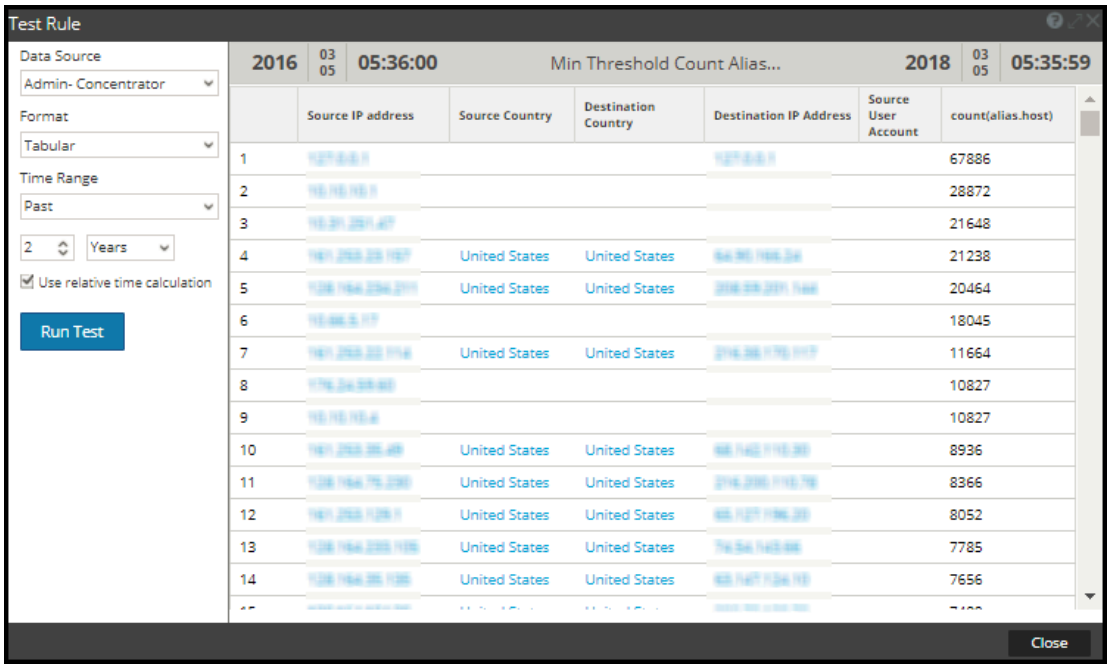
The following figure shows the result without the min_threshold argument. The output results have count of alias.host below 100.



The following figure shows a the min_threshold rule action that sets the minimum limit of 100 on the output. Any output having data less than 100 is not listed.



The following figure shows the result when the min_threshold rule action is applied. Any output having data less than 100 is removed from the result.



regex (string regex, string field)

The regex rule action applies regular expression to the result set. The following is the format of the regex rule action:

regex(regular_expression, meta_name)

Where:

- regular_expression - Regular expression to match the value of the meta.
- meta_name - Meta or field name on which the regex has to be applied.

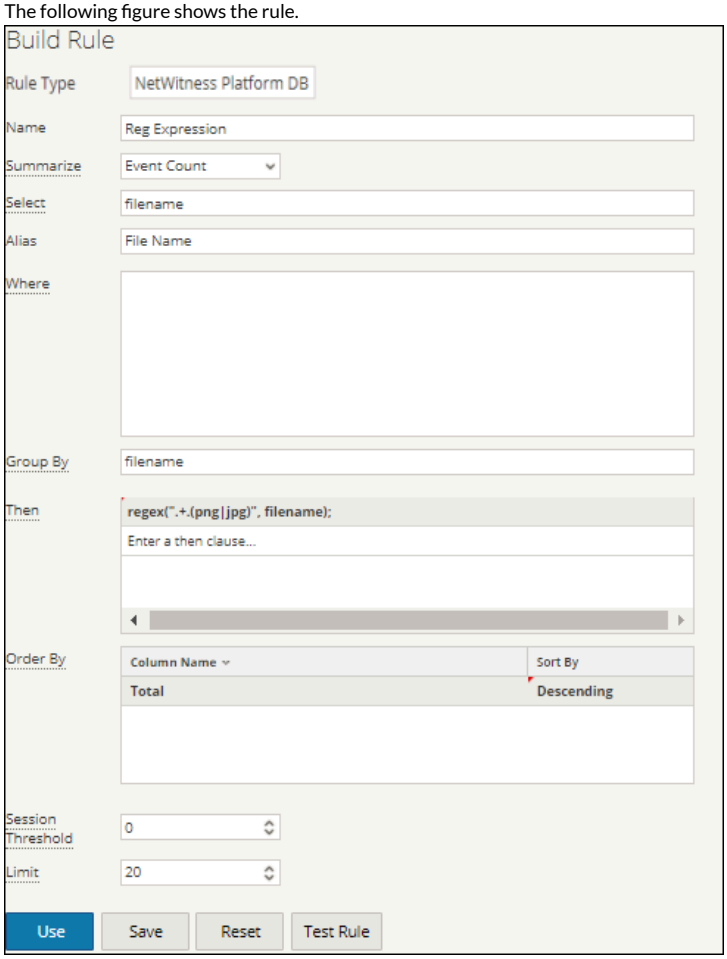
For a comprehensive list of supported regex patterns, refer to http://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html (http://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html).

Sample regex rule action:

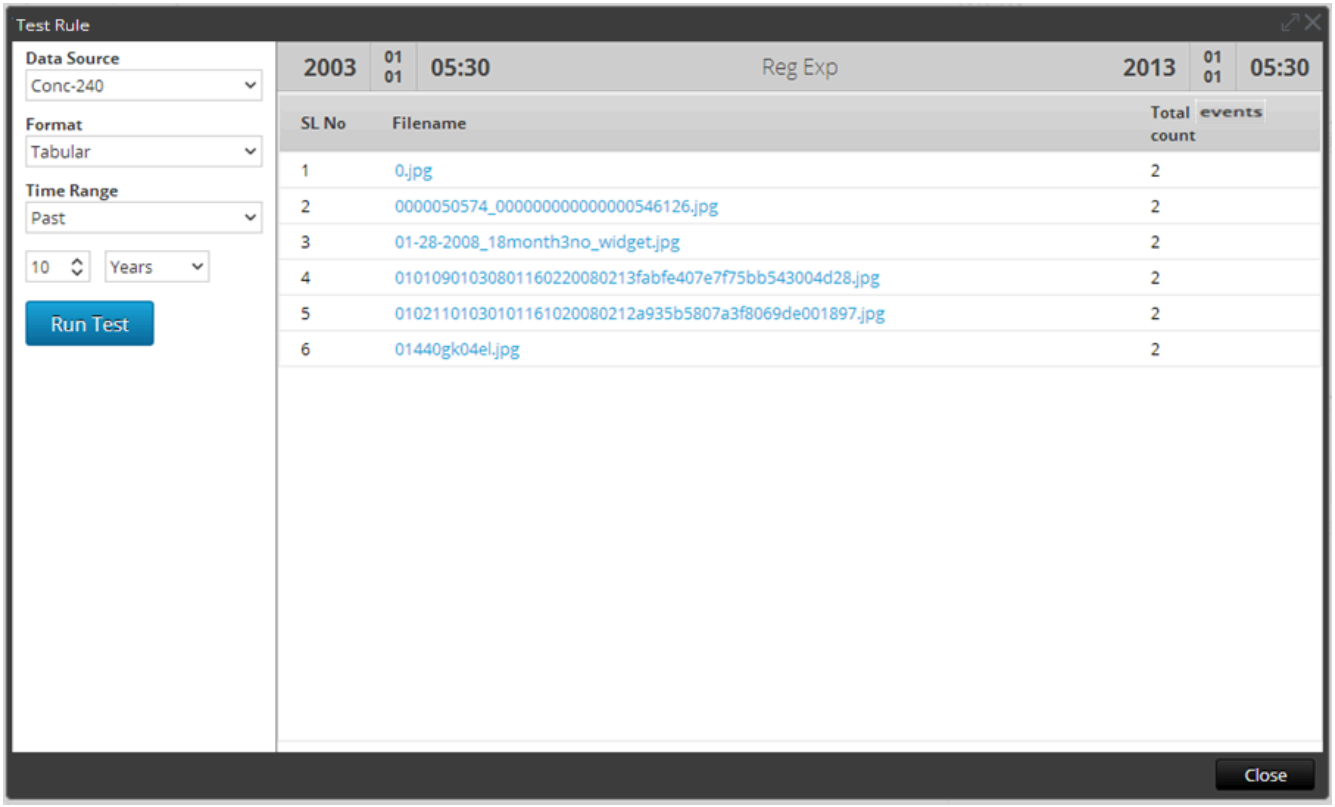
If you want to list filenames of all the PNG and JPEG format files from various sessions, you can write a rule with the following regex rule action:

regex(".+.(png|jpg)", filename);

The following figure shows the rule.



The output with the regex rule action applied is shown in the following figure.

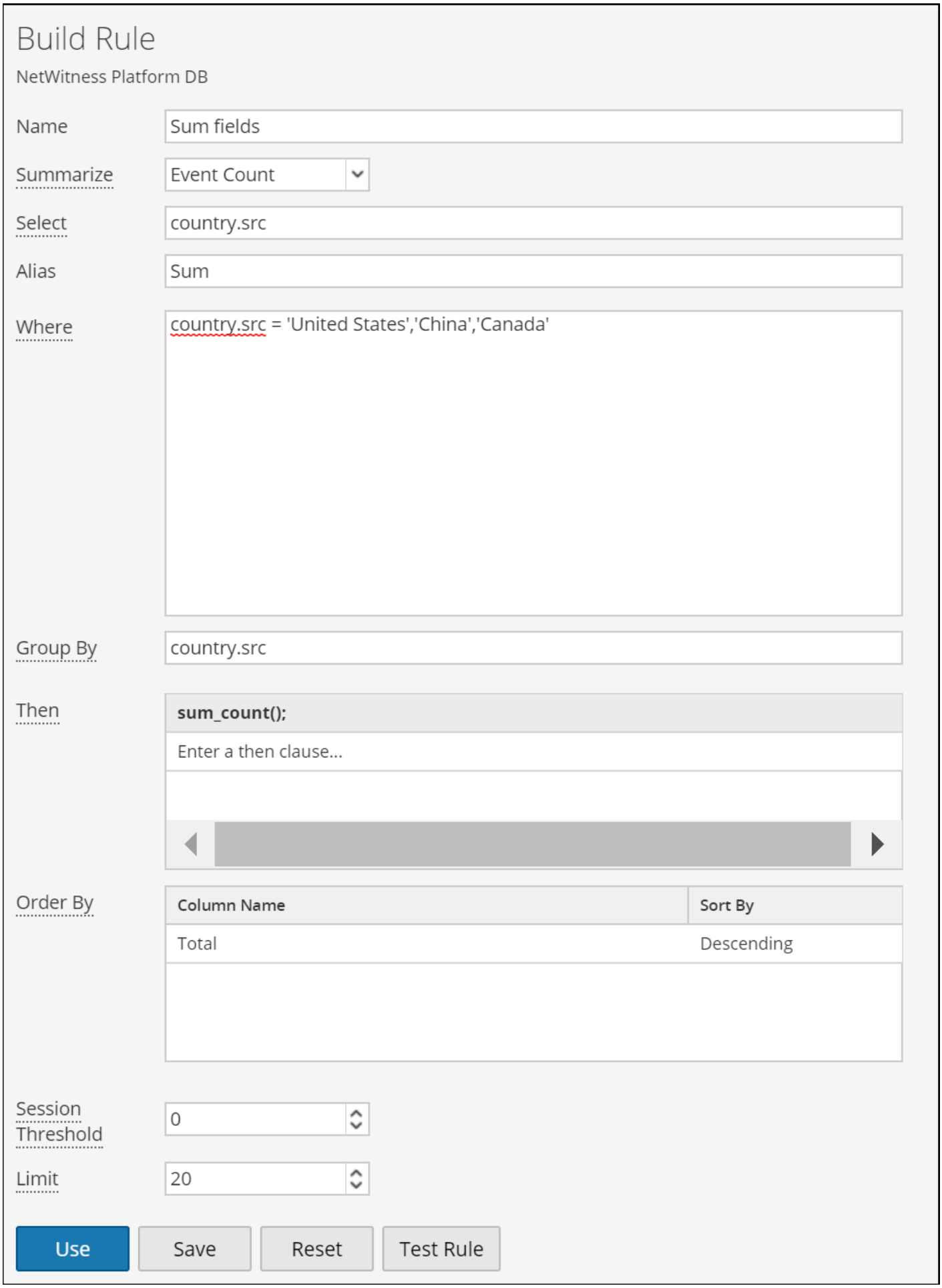


sum_count()

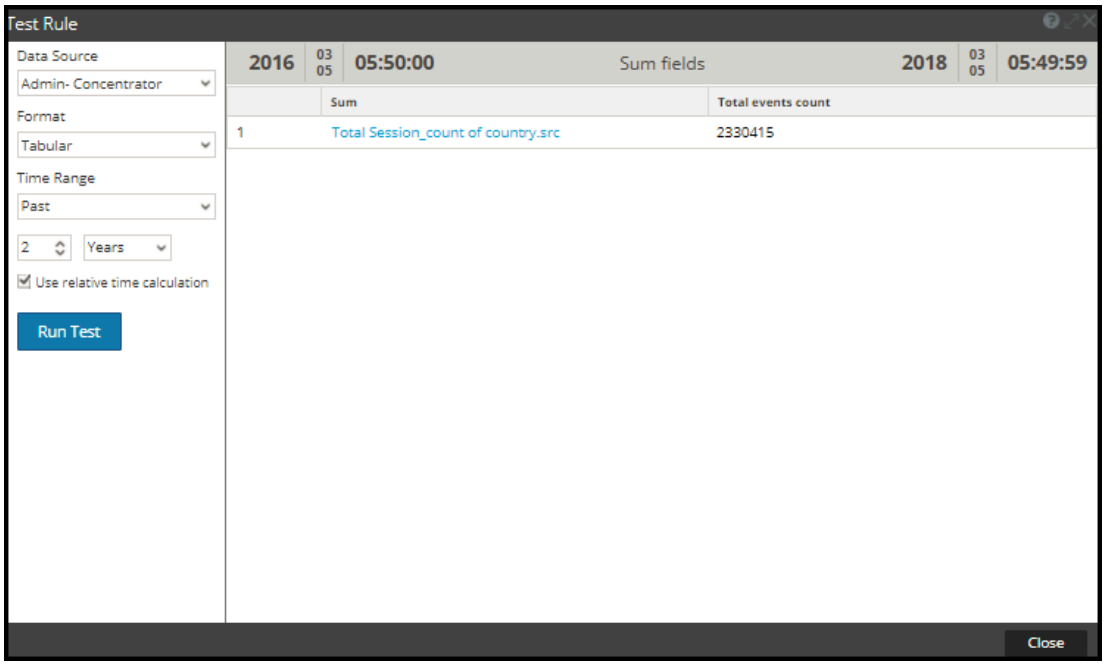
Totals the quantifiers for a given result set. For example, calling a sum_count() for a rule that is sorted by event count totals the size of all values in the result set and displays the total in place of the result set.

Example:

The following figure shows the sum_count() rule action.



With sum_count() rule action, the output shows the total size of all the event counts.

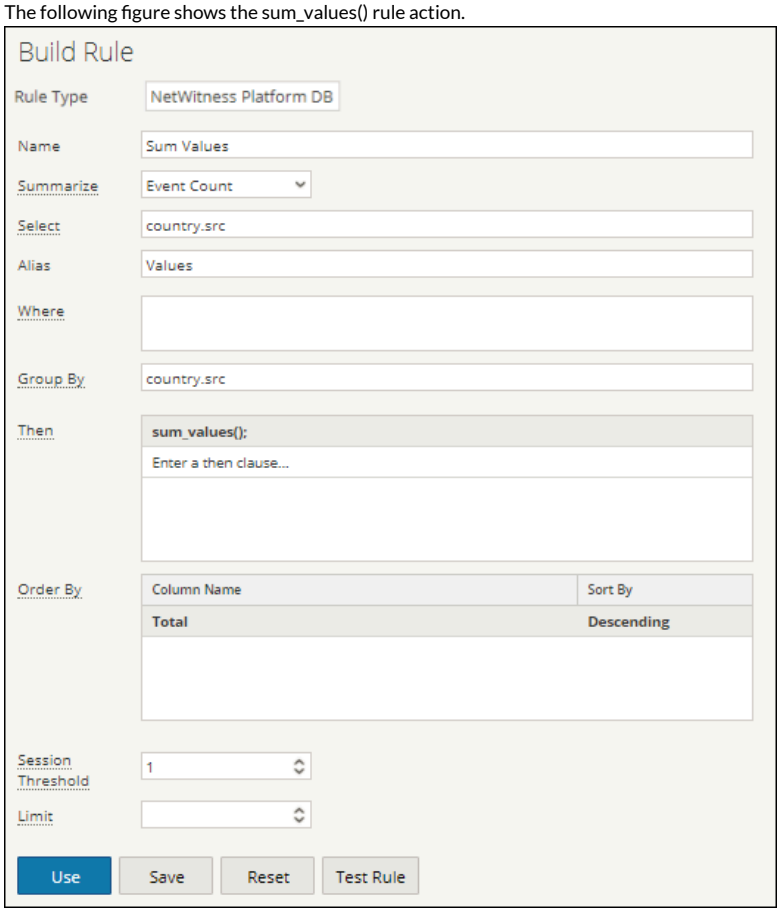


sum_values()

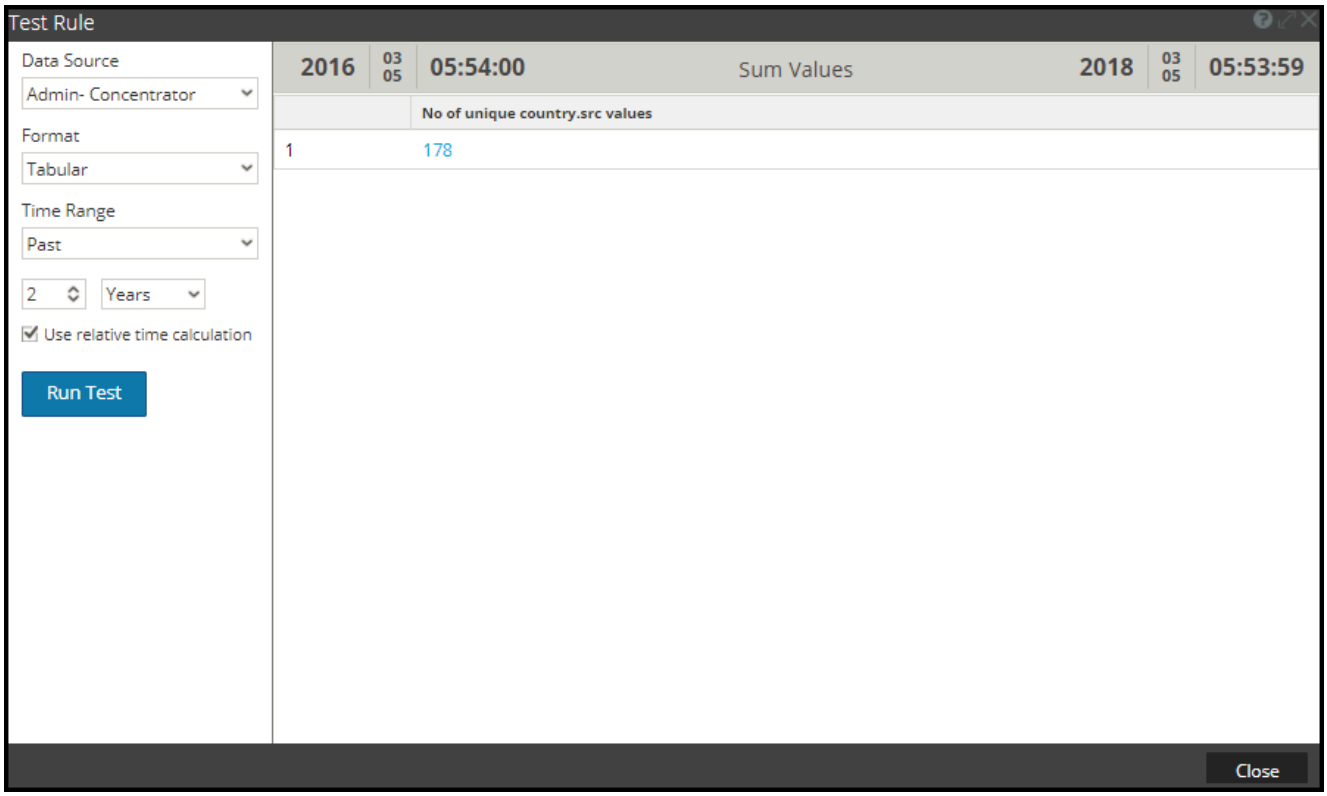
Totals the number of values for a given result set. Use this action to display how many matches exists for a given rule.

Example:

The following figure shows the sum_values() rule action.



The following figure shows the result with sum_values rule action.



show_whats_new()

The show_whats_new() rule action takes any result in a result set and filters out any value that is available in the NetWitness meta database prior to the time frame of the currently running report. When a report runs, NetWitness determines the ID of the first session in the time range of the report. If a value in a result set has a first session id that is greater than the first session id of the report time frame, it did not exist in the NetWitness meta database prior to the report being run and so is new to the NetWitness system relative to the time frame of the report.

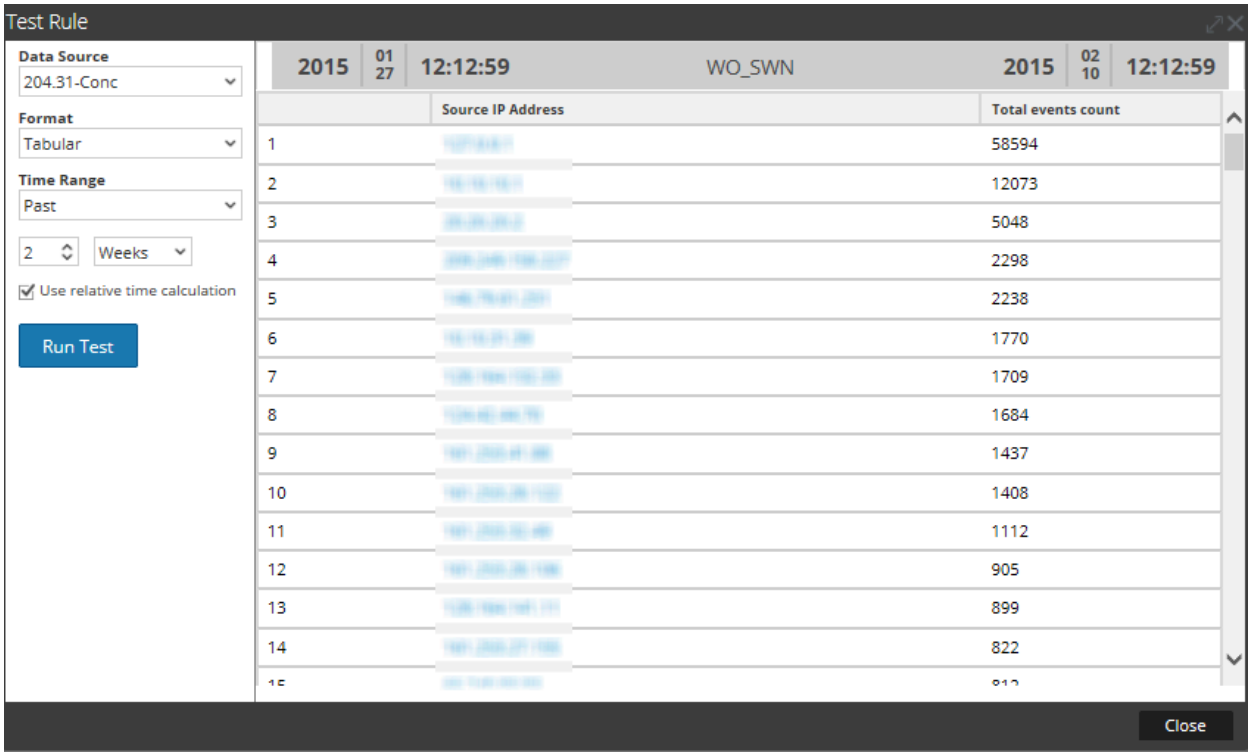
The show_whats_new() rule action is also supported for Custom Aggregate Rule. When multiple meta's are selected in the Custom rule, the first meta is considered for filtering out the old values. See "Example 2" below to understand how this rule action is used for Custom Aggregate Rule.

Note: The show_whats_new() rule action can be used only with an aggregate rule.

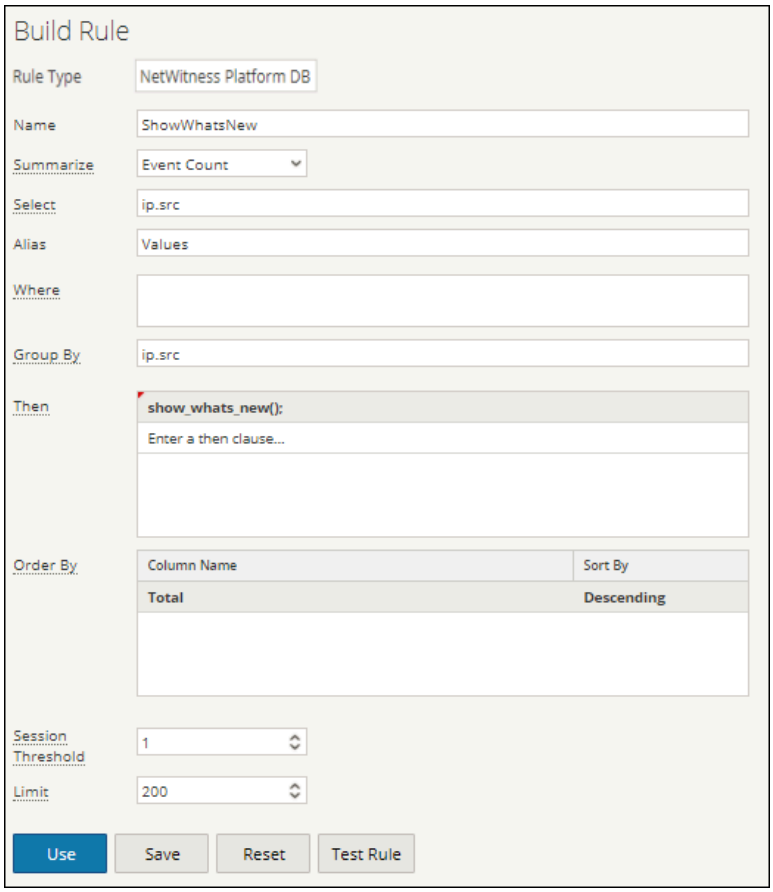
Examples:

1. show_whats_new() for aggregate rule with Event Count

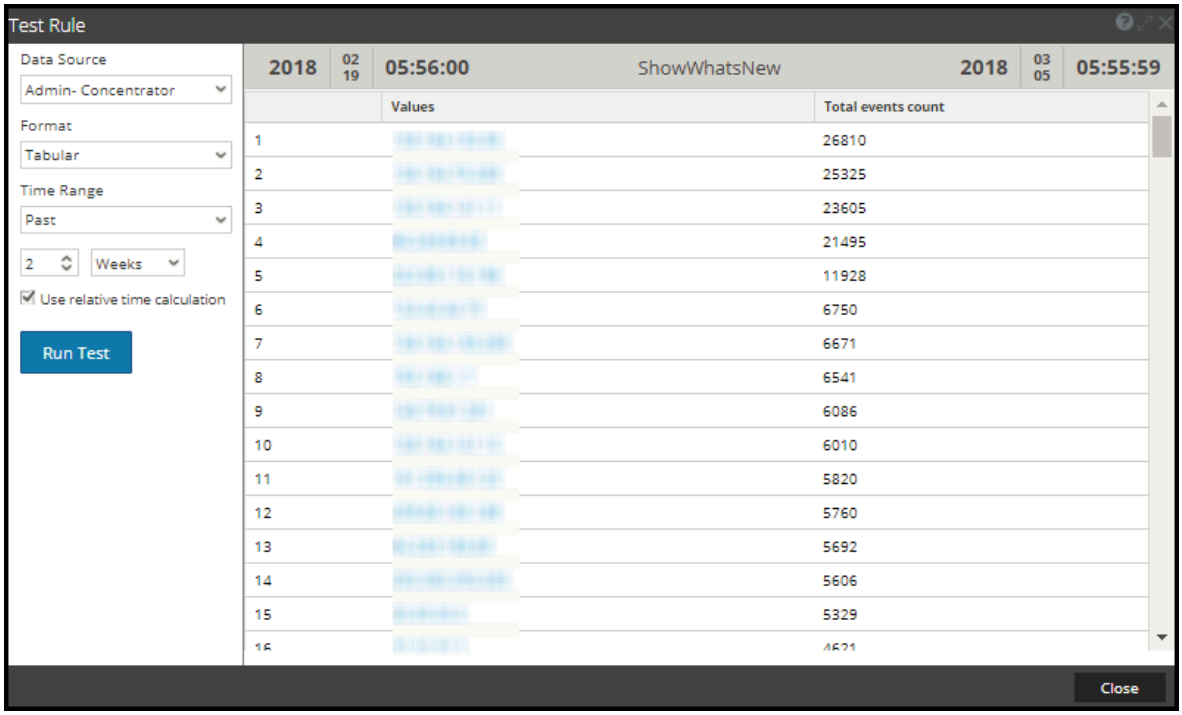
In the following example, all the Source IP Addresses available for the past two weeks are listed.



The following figure shows the use of the show_whats_new rule action to list only the new entries for the past two weeks.

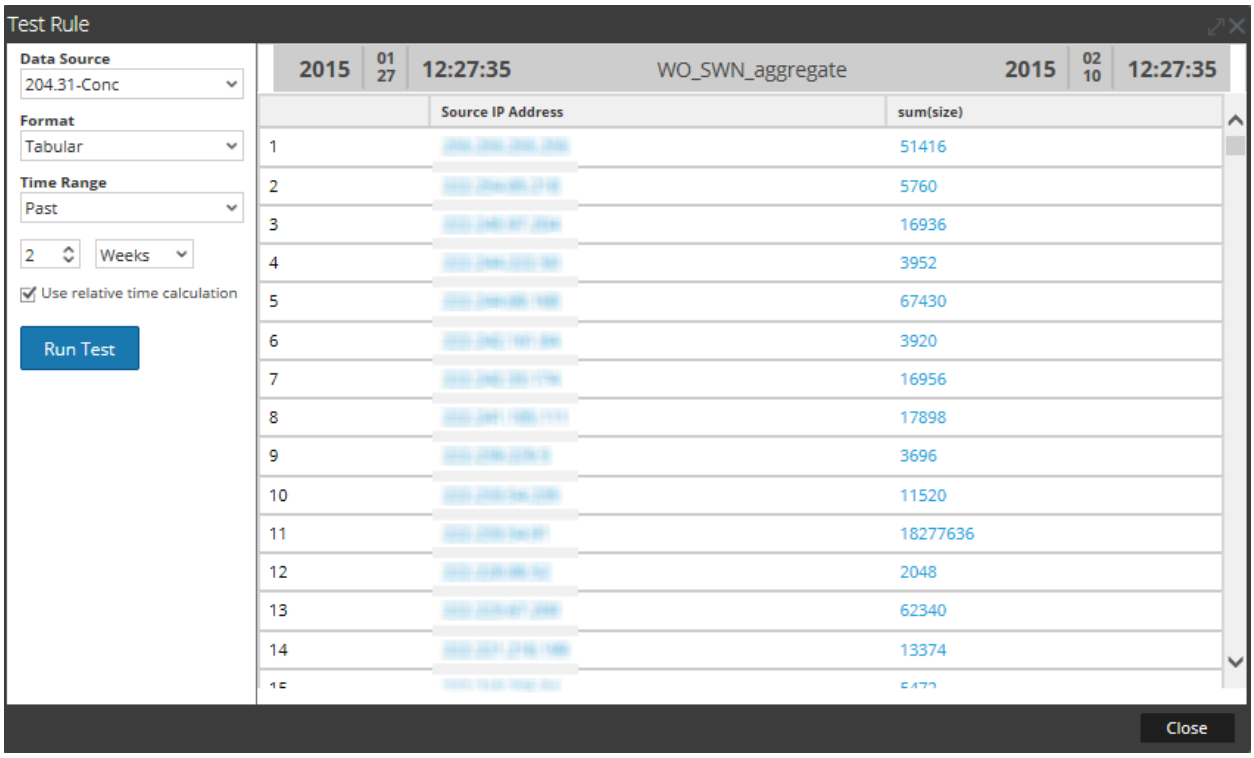


The following figure lists the new entries for the past two weeks.

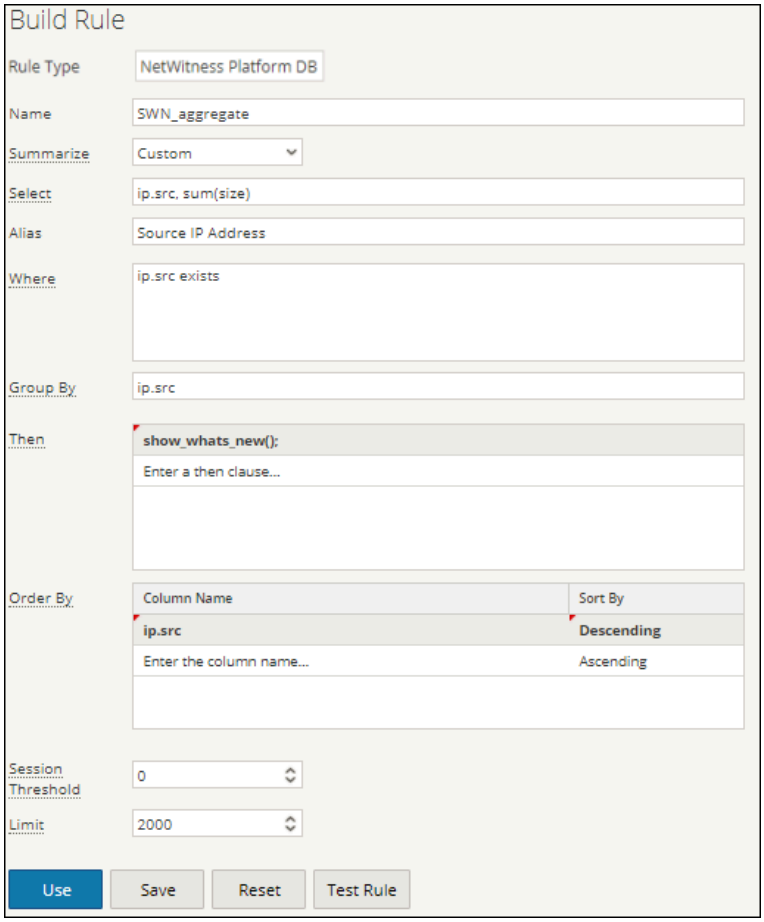


2. show_whats_new() for Custom aggregate rule

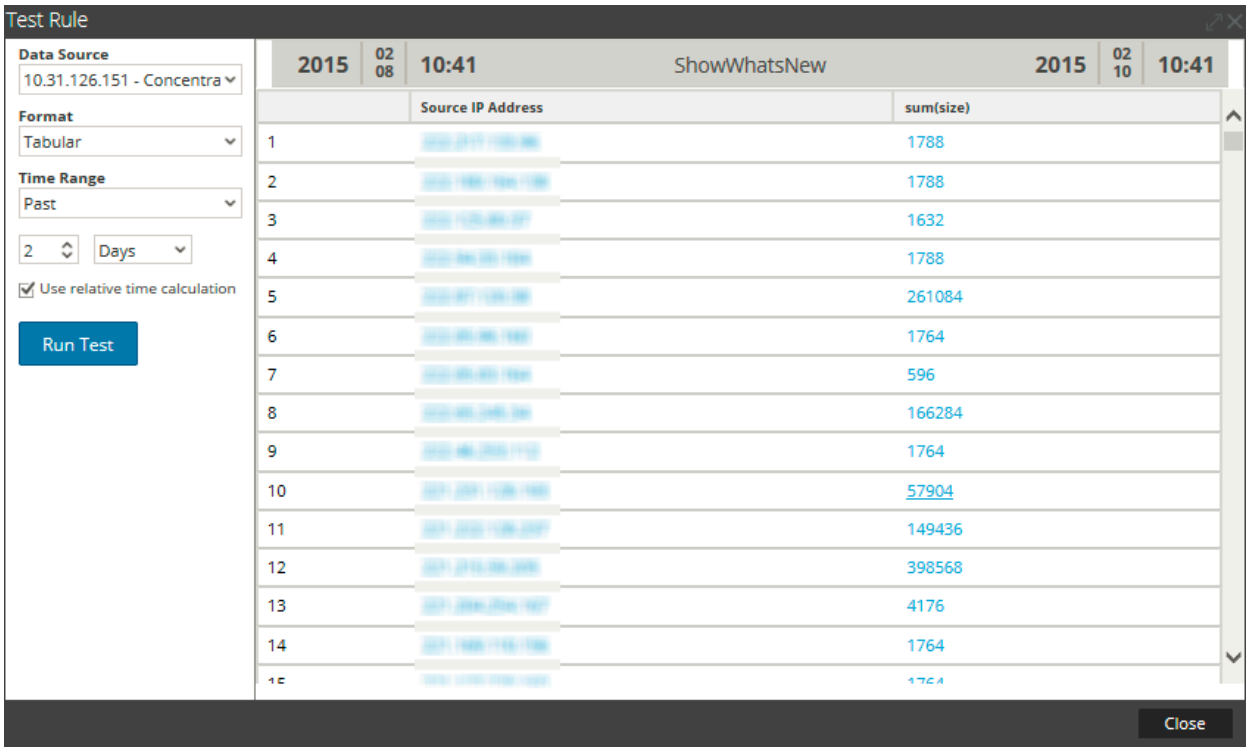
In the following example, all the Source IP Addresses available for the past two weeks are listed.



The following figure shows the use of the show_whats_new rule action to list only the new entries for the past two weeks.



The following figure lists the new entries of Source IP Addresses for the past two weeks.



The power of this feature is that it doesn't matter when the report is run in identifying values that are new to NetWitness. The caveat with this feature is that if a data reset occurs, you will lose your data. However, it is easy to baseline a system and identify changes and new items without a tremendous amount of strain on the system (depending on the size of your result set).

Supported Rule Operators

The NWDB Reporting Engine data source rule syntax supports a subset of rule operators that are supported by NetWitness.

Syntax	Description
*	Use an asterisk (*) as the sole operator in a rule to select all traffic.
=	Equals operator
!=	Does not equal operator
&&	Logical AND operator
||	Logical OR operator
-u	Upper boundary. For example, **tcp.port = 40000-u** selects all TCP ports above 40000.
l-	Lower boundary. For example, **tcp.port = l-40000** selects all TCP ports below 40000.
-	The dash (-) operator only applies to numeric values. Separate the lower and upper boundaries of the range with a dash (-). For example, **tcp.port = 25-443** selects all TCP ports between 25 and 443.

IPDB Rule SyntaxIPDB Rule Syntax

Sample Supported QueriesSample Supported Queries

Respond Rule SyntaxRespond Rule Syntax

The supported rule syntax for the Respond service through descriptions and examples of supported and unsupported syntax. There is a finite set of syntax that you can use to construct rules for reports using the Respond service in this release.

The Reporting Engine supports the following categories of Respond data source rule syntax:

- **select** clause
  - Non-Aggregate Rule
  - Aggregate Rule
- **alias**
- **where** clause
- **where** clause Operators
- Group By
- Order By
- **Limit** field

Note: List is not supported in Respond Data source rules.

Select Clause

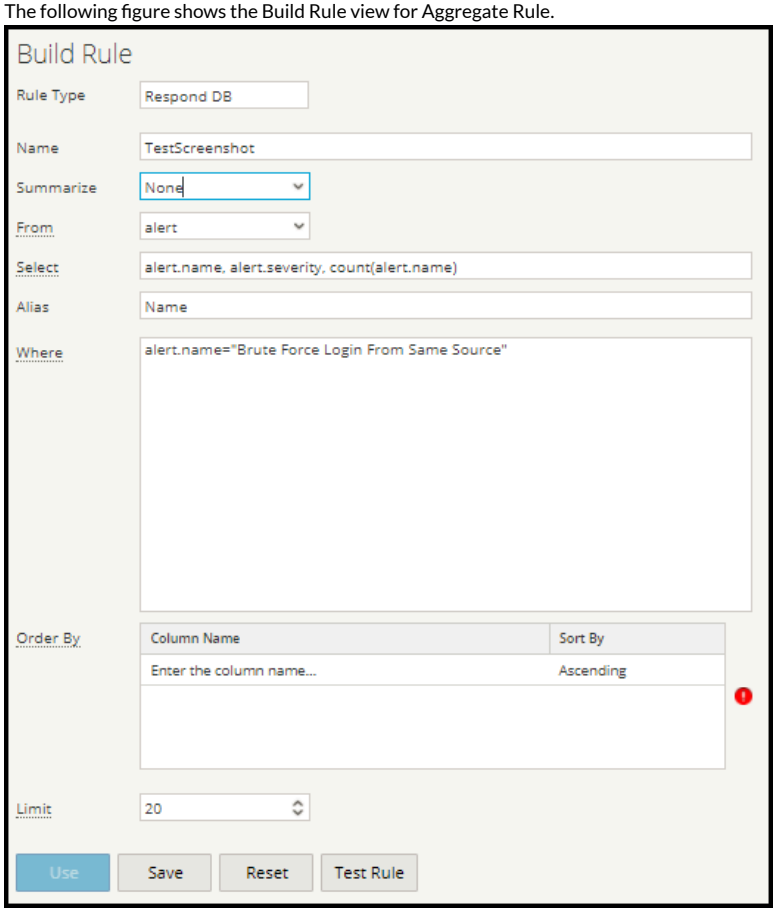
The select clause is a comma separated list of values. For example: select alert.severity, alert.name (https://alert.name), count(*).

There are two types of select clause for Respond Rule:

- Non-aggregate rule

- Aggregate rule

Non-Aggregate Rule
When you want to define a rule without any grouping, choose 'None' in the Summarize field. In a non-aggregate rule, you can select any number of metas in the *Select* clause. For example, select alert.severity, alert.name (https://alert.name).
Aggregate Rule
When you want to query for a specific meta and its associated aggregate value then you must use the Aggregate rule. To get an aggregate, you must choose 'Custom' in the **Summarize** field to include an aggregate function in the *Select* clause. For example, select alert.severity, alert.name (https://alert.name), count(*).
The following figure shows the Build Rule view for Aggregate Rule.



Supported Aggregate Functions
The rules on Respond service supports the following aggregate functions and syntax.

- count
- max
- min
- sum
- avg

Note: The aggregate functions must be added in the end of a select clause for aggregate query. For example, alert.name (https://alert.name), alert.severity, sum(alert.numEvents). By default, a maximum of 10,000 rows results are fetched and this can be configured using the **rsa.response.query.QueryProperties**.
Examples of select Clause Syntax
The following table provides examples of the select Clause Syntax.

Examples	Descriptions
select *column1,column2,column3,...,columnN*	Select specific metas from an Respond Data Source (You must separate each column with a comma.).

Examples of Supported Select Queries
select alert.name (https://alert.name), alert.numEvents, count(alert.numEvents)
select alert.severity, avg(alert.severity)
select alert.timestamp, incidentCreated where alert.timestamp >= 1475658011
Summarize
Summarize determines the type of summarization or aggregation for the rule.

Name	Config Value
Summarize	To query metas without any custom grouping, select: • **None**: To get meta based aggregates, select: • **Custom**: This indicates that expected meta aggregate function is defined in rule select clause.

Alias
Some meta names may not be descriptive, in this case description can be added in the the alias field to make column names more readable. For example, **SELECT**: alert.severity, alert.name (https://alert.name), count(*)
ALIAS: Alert Severity, Alert Name

In the alias field you can enter a name for columns used in the select clause. If you do not specify the alias for one of the field in the select clause, then the default description will be used. For example, if the select clause has Field1,Field2,Field3,Field4, and alias has only Field1, ,Field3,Field4, then for Field2 a default description is used.

Where Clause

The where clause is a language field values and ranges that is used by Respond function. In the where clause, string values have to be enclosed within single quotes.

Examples	Descriptions
alert.host summary ='(Primary) Link status ''Down'' on interface INTNAME.'	For TEXT or string type data, enclose the string or text in single or double quote. If there is any special character such as an apostrophe within the data then you need to add an additional single or double quotes. For example, alert.name (https://alert.name) = 'top alerts from Cote d''Ivoire'.
alert.timestamp >= 1475658011	For Date and Time (date/timestamp data type columns), use the EPOCH syntax.

Supported Where Clause Operators

Operator	Syntax
= (equals)	*column1* = '*value*'
!= (does not equal)	*column1* != '*value*'
>	*column1* > '*value*'
>=	*column1* >= '*value*'
<	*column1* < '*value*'
<=	*column1* <= '*value*'

Group By

Syntax	Function
group by : alert.severity, alert.timestamp, incidentCreated **Note:** Group by field is enabled for Aggregate queries and are not editable.	Respond picks the metas for Group By field from the selected Select clause automatically.

Order By

Order By determines how to sort the result set and is not case sensitive.

Name	Configuration Value
Column Name	The Column Name is the name of the columns by which you want to sort the results. By default, the value is empty. When you click on a column, the value gets populated based on the Summarize field. • order by alert.name (https://alert.name) asc • order by incidentCreated desc • order by count(numEvents) • order by status
Sort By	Sort By determines the order in which you want to sort the results such as ascending or descending. **Note:** For all queries, it is mandatory for you to select the order by field.

Limit field

This indicates the limit to be put on the query while fetching data from the database. If a result set is sorted by event count, packet count, or session size, the limit represents the top (or bottom) N values to be returned. If the result set is not sorted, the first N values are returned.

Previous Page (https://community.netwitness.com/s/article/670015) Next Page (https://community.netwitness.com/s/article/WarehouseDBSimpleRules)

RELATED ARTICLES

Rule Syntax Dialog (/s/article/RuleSyntaxDialog) 1

Understanding and Creating RSA NetWitness Endpoint Alerts in v11.3 (/s/article/UnderstandingandCreatingRSANetWitnessEndpointAlertsinv11-3) 3

Fix Rules with Invalid Syntax (/s/article/FixRuleswithInvalidSyntax) 0

ESA Rule Syntax Error when using the meta "index" to lowercase (/s/article/ESA-Rule-Syntax-Error-when-using-the-meta-index-to-lowercase) 136

How to use the "not begins" operator in Netwitness Reporting Engine query (/s/article/How-to-use-the-not-begins-operator-in-Netwitness-Reporting-Engine-query) 18

Sort by:

Latest Posts

Search this feed...

Nothing here yet?

Log in to post to this feed.

NetWitness Platform (https://www.netwitness.com/)

Acceptable Use Policy (https://www.netwitness.com/wp-content/uploads/NetWitness-Acceptable-Use-Policy-092022.pdf)
Privacy Statement (https://www.netwitness.com/wp-content/uploads/NetWitness-Privacy-Statement-07292024.pdf)

(https://www.linkedin.com/compa
platform (https://twitter.com/NetWitr